Core Java Interview Notes

Object

An object is a fundamental concept in Java that represents a particular instance of a class. It's a real-world entity with state (attributes or fields) and behavior (methods or functions). In Java, objects are created dynamically at runtime. They are instances of a class and occupy memory based on their structure and data.

Class

A class in Java is a blueprint or template for creating objects. It defines the attributes and behavior that objects of the class will have. Classes encapsulate data for the objects and define methods to operate on that data. They serve as the foundation for object-oriented programming in Java.

Interface

An interface in Java is a reference type that defines a contract for classes to follow. It contains only method signatures (without implementations), default methods, static methods, and constant variables.Interfaces are used to achieve abstraction and multiple inheritance in Java.Classes implement interfaces by providing concrete implementations for all the methods defined in the interface.

Constructor

A constructor in Java is a special type of method that is automatically called when an object of a class is instantiated or created.It has the same name as the class and does not have a return type, not even void. The purpose of a constructor is to initialize the newly created object, often by assigning initial values to instance variables or performing other setup tasks.

Static Method

A static method in Java belongs to the class rather than any specific instance of the class. It can be called directly using the class name without creating an object of the class. Static methods are commonly used for utility functions or operations that don't require access to instance variables.

Non-Static Method

A non-static method in Java is associated with instances of the class. It operates on the specific instance's data and can access instance variables and other non-static members of the class. Non-static methods are invoked on objects of the class and are used to perform actions or operations specific to those objects.

Abstract Class

An abstract class in Java is a class that cannot be instantiated on its own and may contain abstract methods (methods without a body) along with concrete methods.Abstract classes are used to provide a common base implementation for its subclasses while allowing certain methods to be overridden by subclasses.Abstract classes may also contain instance variables, constructors, and other methods like a regular class.

Composition:

Without existing container object there is no chance of existing contained objects.Container and contained objects are strongly associated and this strong association is known as composition.Composition represents a stronger form of association where the contained objects are integral parts of the container object.In composition, if the container object is destroyed, all the contained objects are also destroyed.Composition is represented by a stronger relationship than aggregation and is often implemented by creating instances of other classes within the container class.

JVM, JRE, and JDK :

JVM (Java Virtual Machine) ::

- JVM stands for Java Virtual Machine. It is an abstract computing machine that enables a computer to run Java programs.
- JVM interprets compiled Java bytecode and executes it on the native hardware.
- JVM provides platform independence by allowing Java programs to run on any device or operating system that has a compatible JVM installed.
- JVM manages memory allocation and garbage collection.

JRE (Java Runtime Environment)::

- JRE stands for Java Runtime Environment. It is a software package that includes the JVM, libraries, and other components necessary to run Java applications.
- JRE provides the environment required for executing Java applications**. It includes the JVM along with class libraries** and other supporting files.
- End-users who want to run Java applications need only the JRE installed on their systems.
- JRE does not contain development tools such as compilers and debuggers.

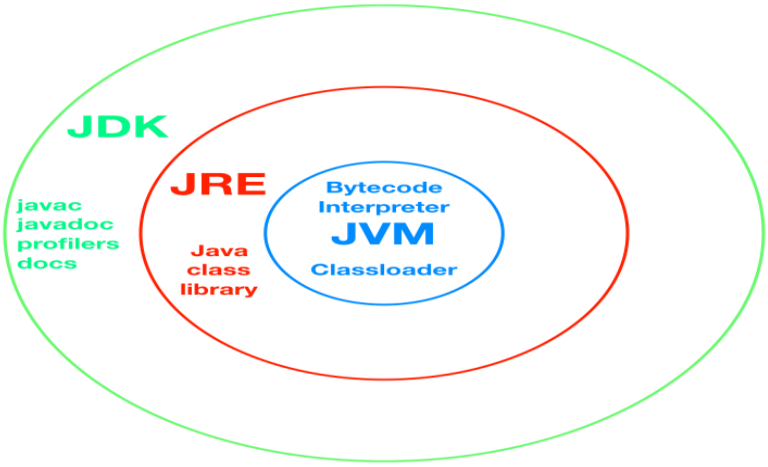
JDK (Java Development Kit)

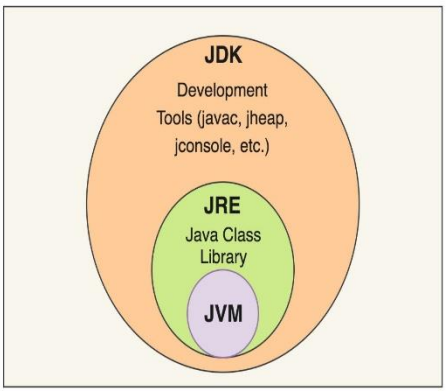
- JDK stands for Java Development Kit. It is a software development kit that provides tools and libraries for developing Java applications.
- **JDK includes the JRE, development tools (such as compiler and debugger), and additional libraries** necessary for developing Java applications.
- Developers use JDK to write, compile, debug, and run Java programs.

Relationship

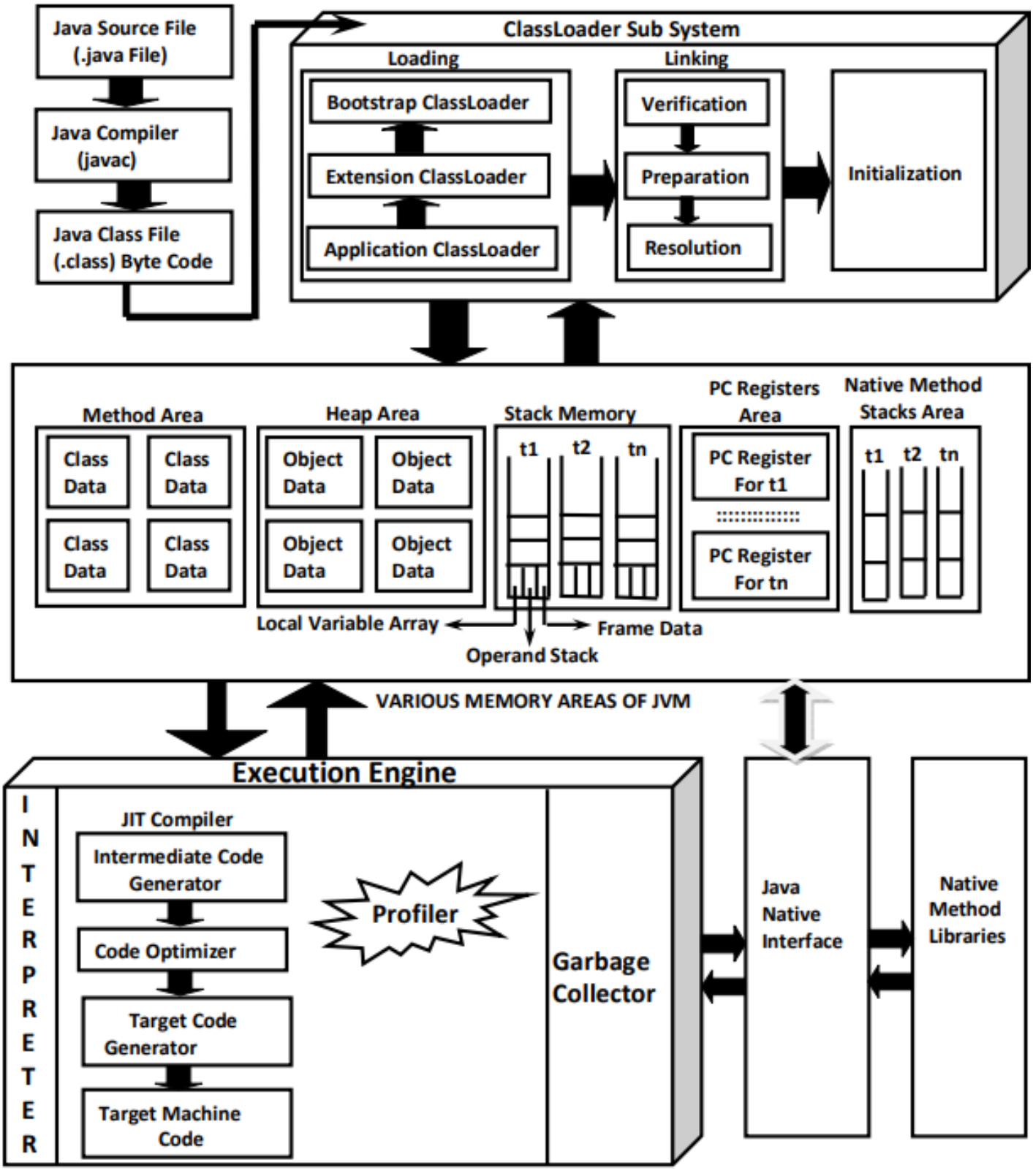
Hierarchy : JDK includes the JRE, and the JRE includes the JVM.

Development vs. Runtime : JDK is used for Java application development, while JRE is used for running Java applications. Both JDK and JRE rely on the JVM to execute Java bytecode.





JVM Architecture



1) Loading:

- Loading Means Reading Class Files and Store Corresponding Binary Data in Method Area.
- For Each Class File JVM will Store the following Information in **Method Area**.
  - Fully Qualified Name of the Loaded Class OR Interface OR enum.
  - Fully Qualified Name of its Immediate Parent Class.

- Whether .class File is related to Class OR Interface OR enum.
- The Modifiers Information
- Variables OR Fields Information
- Methods Information
- Constant Pool Information and so on.

2) Linking:

- Linking Consists of 3 Activities
  - Verification
  - Preparation
  - Resolution

Verification:

- It is the Process of ensuring that Binary Representation of a Class is Structurally Correct OR Not.
- That is JVM will Check whether .class File generated by Valid Compiler OR Not.i.e whether .class File is Properly Formatted OR Not.
- Internally Byte Code Verifier which is Part of ClassLoader Sub System is Responsible for this Activity.
- If Verification Fails then we will get Runtime Exception Saying java.lang.VerifyError.

Preparation:

- In this Phase JVM will Allocate Memory for the Class Level Static Variables and Assign DefaultValues (But Not Original Values).
- **Note**: Original Values will be assigned in Initialization Phase.

Resolution:

- It is the Process of Replaced Symbolic References used by the Loaded Type with Original References.
- Symbolic References are Resolved into Direct References by searching through Method Area to Locate the Referenced Entity.

3) Initialization:

- In this Phase All Static Variables will be assigned with Original Values and Static Blocks will be executed from fromtop to bottom and from Parent to Child.

Types of ClassLoaders:

- Every ClassLoader Sub System contains the following 3 ClassLoaders.
  1. BootstrapClassLoader OR PrimordialClassLoader
  2. ExtensionClassLoader
  3. ApplicationClassLoader OR SystemClassLoader

BootstrapClassLoader

- This ClassLoader is Responsible to load classes from **jdk\jre\lib** folder.
- All core java API classes present in rt.jar which is present in this location only. Hence all API classes (like String, StringBufferetc) will be loaded by Bootstrap class Loader only.
- That is BootstrapClassLoader is Responsible to Load Classes from BootstrapClassPath.
- BootstrapClassLoader is by Default Available with the JVM.

Extension ClassLoader:

- It is the Child of Bootstrap ClassLoader.
- ThisClassLoader is Responsible to Load Classes from Extension Class Path i.e **jdk\jre\lib\ext**
- This ClassLoader is implemented in Java and the corresponding .class File Name is sun.misc.Launcher$ExtClassLoader.class

Application ClassLoader OR System ClassLoader:

- It is the Child of Extension ClassLoader.
- This ClassLoader is Responsible to Load Classes from Application Class Path (**Current Working Directory**).
- It Internally Uses Environment Variable Class Path.
- Application ClassLoader is implemented in Java and the corresponding .class File Name is sun.misc.Launcher$appClassLoader.class

How Java ClassLoader Works?

- ClassLoader follows Delegation Hierarchy Principle.
- Whenever JVM Come Across a Particular Class, first it will Check whether the corresponding Class is Already Loaded OR Not.
- If it is Already Loaded in Method Area then JVM will Use that Loaded Class.
- If it is Not Already Loaded then JVM Requests ClassLoader Sub System to Load that Particular Class.
- Then ClassLoaderSub System Handovers the Request to ApplicationClassLoader.
- ApplicationClassLoader Delegates that Request to ExtensionClassLoader and ExtenstionClassLoader in-turn Delegates that Request to BootstrapClassLoader.
- BootstrapClassLoader Searches in Bootstrap Class Path for the required .class File i.e (jdk/jre/lib)
- If the required .class is Available, then it will be Loaded. Otherwise BootstrapClassLoader Delegates that Request to ExtensionClassLoader .
- ExtensionClassLoader will Search in Extension Class Path (jdk/jre/lib/ext). If the required .class File is Available then it will be Loaded, Otherwise it Delegates that Request to ApplicationClassLoader.
- ApplicationClassLoader will Search in Application Class Path (Current Working Directory). If the specified .class is Already Available, then it will be Loaded.
- Otherwise we will get Runtime Exception Saying ClassNotFoundException OR NoClassDefFoundError.

JVM Memomry Management

- JVM (Java Virtual Machine) memory management involves the allocation, usage, and deallocation of memory resources within the Java runtime environment. It is responsible for managing the memory required by Java applications and ensuring efficient utilization of system resources.
- While Loading and Running a Java Program JVM required Memory to Store Several Things Like Byte Code, Objects, Variables, Etc.
- Total JVM Memory organized in the following 5 Categories:
    - Method Area
    - Heap Area OR Heap Memory
    - Java Stacks Area
    - PC Registers Area
    - Native Method Stacks Area

Method Area (PermGen/Metaspace)

- This memory area stores class metadata, method information, and static variables. In older versions of Java, it was known as the Permanent Generation (PermGen), but in newer versions, it's referred to as Metaspace. Metaspace dynamically adjusts its size based on the application's needs and is not subject to the same limitations as PermGen.
- Method Area will be Created at the Time of JVM Start- Up.
- It will be Shared by All Threads (Global Memory).
- This Memory Area Need Not be Continuous.
- Method area shows runtime constant pool.
- Total Class Level Binary Information including Static Variables Stored in Method Area .

Heap Area OR Heap Memory

- Programmer Point of View Heap Area is Consider as Important Memory Area.
- Heap Area will be Created at the Time of JVM Start- Up.
- Heap Area can be accessed by All Threads (Global OR Sharable Memory).
- Heap Area Need Not be Continuous.
- All Objects and corresponding Instance Variables will be stored in the Heap Area.
- Every Array in Java is an Object and Hence Arrays Also will be stored in Heap Memory Only.

Stack Memory:

- Each thread in a Java application has its own stack memory, used for storing method invocations, local variables, and partial results.
- For Every Thread JVM will Create a Separate Runtime Stack.
- Runtime Stack will be Created Automatically at the Time of Thread Creation.
- All Method Calls and corresponding Local Variables, Intermediate Results will be stored in the Stack.

- For Every Method Call a Separate Entry will be Added to the Stack and that Entry is Called Stack Frame OR Activation Record.
- After completing that Method Call the corresponding Entry from the Stack will be Removed.
- After completing All Method Calls, Just Before terminating the Thread,the Runtime Stack will be destroyed by the JVM.
- The Data stored in the Stack can be accessed by Only the corresponding Thread and it is Not Available to Other Threads .

PC (Program Counter) Registers Area:

- For Every Thread a Separate PC Register will be Created at the Time of Thread Creation.
- PC Registers contains Address of Current executing Instruction.
- Once Instruction Execution Completes Automatically PC Register will be incremented to Hold Address of Next Instruction.

Native Method Stacks:

- For Every Thread JVM will Create a Separate Native Method Stack.
- All Native Method Calls invoked by the Thread will be stored in the corresponding Native Method Stack .

- **Note :** Static Variables will be stored in Method Area whereas Instance Variables will be stored in Heap Area and Local Variables will be stored in Stack Area.

Method Area	Heap Memory	Stack Memory
• **Class metadata (method data, field data, runtime constant pool)** • **Static variables** • **Static methods** • **Instance methods** • **Constructor code**	• **Objects** • **Instance variables (fields of objects)** • **Arrays** • **Strings created using new keyword and interned strings**	• **Local variables** • **Method parameters** • **Intermediate results** • **Method call frames (including local variables, operand stack, and return address)**

Garbage Collection

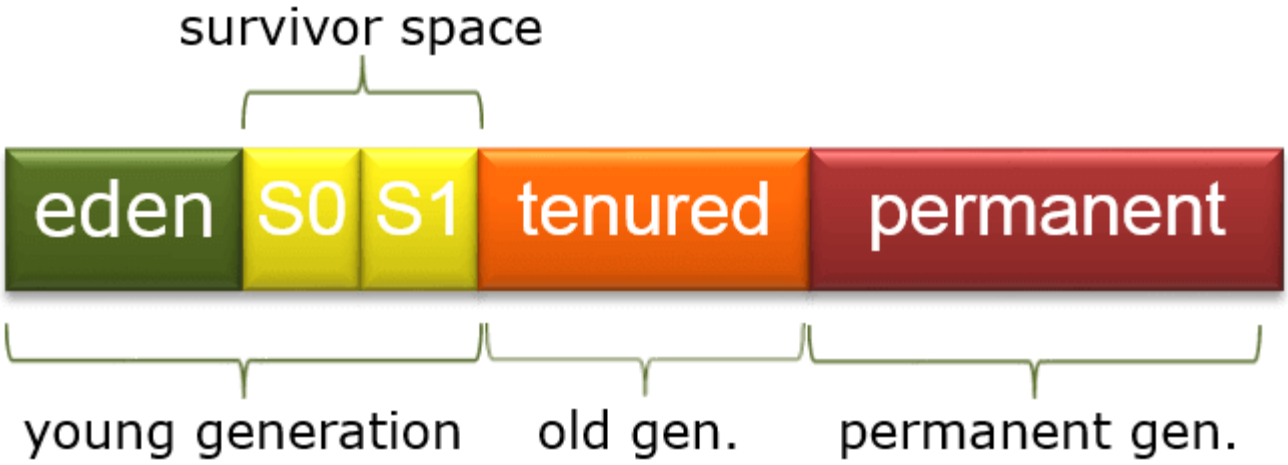
Garbage collection (GC) in Java is an automated process that manages memory by reclaiming memory used by objects that are no longer needed or reachable by the program, ensuring efficient memory usage and preventing memory leaks.

- The way to make an object eligible for GC
  - Nullifying the reference variable
  - Reassign the reference variable
  - Objects created inside a method
  - Island of Isolation
- The methods for requesting JVM to run GC

- By System class
- By Runtime class

➢ Finalization

- Case 1 : Just before destroying any object GC calls finalize() method on the object
- Case 2 : We can call finalize() method explicitly
- Case 3 : finalize() method can be call either by the programmer or by the GC
- Case 4 : On any object GC calls finalize() method only once Memory leaks



Interpretation and Just-In-Time (JIT)

- The process of converting Java bytecode into machine code that can be executed by the underlying hardware involves both interpretation and Just-In-Time (JIT) compilation.
- By combining interpretation and JIT compilation, Java achieves a balance between platform independence and performance.
- Interpretation ensures that Java bytecode can run on any platform with a compatible JVM, while JIT compilation optimizes performance by translating frequently executed bytecode into native machine code.

Interpreter:

- When you run a Java program, the Java Virtual Machine (JVM) loads the bytecode generated by the Java compiler. The interpreter component of the JVM reads this bytecode line by line and executes the corresponding machine-level instructions. This interpretation process ensures that the Java program can run on any platform with a compatible JVM, as the bytecode is platform-independent.

Just-In-Time (JIT) Compilation:

- While interpretation allows for platform independence, it can lead to slower execution compared to native code execution. To address this, modern JVM implementations typically employ JIT compilation. The JIT compiler analyzes the bytecode as it is being executed and identifies portions of code (hot spots) that are frequently executed. It then compiles these portions of code into native machine code, optimizing them for the specific hardware and caching the compiled code for future use.

What Is a Memory Leak?

- A Memory Leak is a situation where there are objects present in the heap that are no longer used, but the garbage collector is unable to remove them from memory, and therefore, they're unnecessarily maintained.
- A memory leak is bad because it blocks memory resources and degrades system performance over time. If not dealt with, the application will eventually exhaust its resources, finally terminating with a fatal java.lang.OutOfMemoryError.

- There are two different types of objects that reside in Heap memory, referenced and unreferenced. Referenced objects are those that still have active references within the application, whereas unreferenced objects don't have any active references.
- The garbage collector removes unreferenced objects periodically, but it never collects the objects that are still being referenced. This is where memory leaks can occur.
- Memory leaks in Java typically occur when objects are unintentionally retained in memory longer than necessary, preventing the Java garbage collector from reclaiming memory occupied by those objects.

Symptoms of a Memory Leak

- Severe performance degradation when the application is continuously running for a long time.
- OutOfMemoryError heap error in the application.
- Spontaneous and strange application crashes.
- The application is occasionally running out of connection objects.

Types of Memory Leaks in Java

1. Memory Leak Through static Fields

- The first scenario that can cause a potential memory leak is heavy use of static variables.
- In Java, static fields have a life that usually matches the entire lifetime of the running application (unless ClassLoader becomes eligible for garbage collection).

2. Through Unclosed Resources

- Whenever we make a new connection or open a stream, the JVM allocates memory for these resources. A few examples of this include database connections, input streams, and session objects.
- Forgetting to close these resources can block the memory, thus keeping them out of the reach of the GC. This can even happen in case of an exception that prevents the program execution from reaching the statement that's handling the code to close these resources.

3. Improper equals() and hashCode() Implementations

- When defining new classes, a very common oversight is not writing proper overridden methods for the equals() and hashCode() methods.
- HashSet and HashMap use these methods in many operations, and if they're not overridden correctly, they can become a source for potential memory leak problems.

4. Inner Classes That Reference Outer Classes

- This happens in the case of non-static inner classes (anonymous classes). For initialization, these inner classes always require an instance of the enclosing class.
- Every non-static Inner Class has, by default, an implicit reference to its containing class. If we use this inner class' object in our application, then even after our containing class' object goes out of scope, it won't be garbage collected.

How to Prevent It?

- Migrating to the latest version of Java that uses modern Garbage Collectors such as ZGC that uses root references to find unreachable objects. Since the references are found from the root, this will solve the cyclic problem, like the anonymous class holding reference to the container class.

5. Through finalize() Methods

- Use of finalizers is yet another source of potential memory leak issues. Whenever a class' finalize() method is overridden, then objects of that class aren't instantly garbage collected. Instead, the GC queues them for finalization, which occurs at a later point in time.
- Additionally, if the code written in the finalize() method isn't optimal, and if the finalizer queue can't keep up with the Java garbage collector, then sooner or later our application is destined to meet an OutOfMemoryError.

Package

- A package in Java is used to group related classes. Think of it as a folder in a file directory.
- A package is a way to organize classes and interfaces into namespaces.
- It helps in avoiding naming conflicts and provides a mechanism for access control.

Namespace:

- Namespace means that classes and interfaces within a package are uniquely identified by their fully qualified name, which includes the package name.

Access Control:

- Classes and interfaces declared with the public access modifier are accessible from other packages, while those without the public modifier are only accessible within the same package (package-private access).

Import Statements:

- To use classes and interfaces from other packages, you can use import statements. Import statements allow you to refer to classes and interfaces by their simple names instead of their fully qualified names.

Is-A Relationship ::

- The "is-a" relationship, also known as inheritance, represents a relationship where one class is a specialized version of another class.
- This relationship is typically expressed using subclass and superclass relationships.

For example:

- A `Car` **is-a** `Vehicle`.

- A `Rectangle` **is-a** `Shape`.

In Java, this relationship is implemented using the `extends` keyword to create subclasses.

```
class Vehicle {
   // Vehicle class definition
}

class Car extends Vehicle {
   // Car class definition
}
```

Has-A Relationship ::

- The "has-a" relationship represents a relationship where one class contains another class as a member.
- This implies that one class has a reference to another class or holds an instance of another class.

For example:

- A `Car` **has-a** `Engine`.

- A `House` **has-a** `Room`.

```
class Engine {
   // Engine class definition
}

class Car {
   private Engine engine;  // Car has-a Engine
   // Constructor, methods, etc.
}
```

- Has-a relationship is also known as composition or aggregation .
- There is no specific keyword is there to represent has-a relationship ,but most of the time we depends on **new** keyword.
- The main advantage of has-a relationship is reusability of code.

Composition:

- Without existing container object there is no chance of existing contained objects.
- Container and contained objects are strongly associated and this strong association is known as composition.
- Composition represents a stronger form of association where the contained objects are integral parts of the container object.
- In composition, if the container object is destroyed, all the contained objects are also destroyed.
- Composition is represented by a stronger relationship than aggregation and is often implemented by creating instances of other classes within the container class.

Example:

University consists of several dept. without existing universiy there is no chance of existing department.

Aggregation:

- Without existing container object there is a chance of existing contained objects.
- Container & Contained objects are weakly associated and this weak association is known as aggregation.
- Aggregation represents a "has-a" relationship between classes, where one class contains references to objects of another class. It's a weaker form of association compared to composition.
- In aggregation, the contained objects can exist independently of the container object. They have their own lifecycle, and if the container object is destroyed, the contained objects can still exist.
- Aggregation is represented by a unidirectional association between classes, often implemented using instance variables.

Example:

Dept. consist of several proffesor without existing dept. there may be chance of existing proffesor.

Key Differences

- **Is-A Relationship**: Represents inheritance and is used for modeling specialization and generalization relationships. It defines a hierarchy where subclasses inherit attributes and behaviors from superclasses.
- **Has-A Relationship**: Represents composition and is used for modeling containment relationships. It defines how one class is composed of or contains another class as a part of its structure.

Wrapper classes

- A Wrapper class in Java is a class whose object wraps or contains primitive data types. When we create an object to a wrapper class, it contains a field and in this field, we can store primitive data types. In other words, we can wrap a primitive value into a wrapper class object.
- **This is particularly useful when you need to pass primitive types as arguments to methods that require objects.**
- Primitive data types cannot be `null`, but sometimes you need a variable that can be `null`. Wrapper classes provide a way to represent `null` values for primitive types by allowing them to be `null` as objects.
- Generics in Java do not work with primitive types. Wrapper classes enable you to use collections and other generic types with primitive data types by providing generic classes that correspond to each primitive type (`Integer` for `int`, `Double` for `double`, etc.)

For example, to convert an `int` to an `Integer`:

int primitiveInt = 10;

Integer wrapperInt = Integer.valueOf(primitiveInt); // Converting int to Integer

And to convert it back:

int convertedInt = wrapperInt.intValue(); // Converting Integer to int

****Interview questions about wrapper classes****

1. What is a wrapper class in Java?
   - A wrapper class in Java is a class that provides an object representation of primitive data types. It allows primitive data types to be used as objects in Java applications.

2. Why do we need wrapper classes in Java?
   - Wrapper classes are needed in Java because some situations, such as working with collections, require objects instead of primitive data types. Additionally, wrapper classes provide utility methods to work with primitive types and enable null values to be assigned.

3. Name the eight primitive data types in Java and their corresponding wrapper classes.
   - Primitive data types: byte, short, int, long, float, double, char, boolean
   - Wrapper classes: Byte, Short, Integer, Long, Float, Double, Character, Boolean

4. How do you convert a primitive data type to its corresponding wrapper class in Java?

- You can use the constructors provided by the wrapper classes to convert primitive data types to their corresponding wrapper class objects. For example:

```
int primitiveInt = 10;
Integer wrapperInt = new Integer(primitiveInt);
```

5. How do you convert a wrapper class object to its corresponding primitive data type in Java?
- You can use methods such as intValue(), doubleValue(), floatValue(), etc., provided by the wrapper classes to convert wrapper objects to primitive data types. For example:

```
Integer wrapperInt = Integer.valueOf(10);
int primitiveInt = wrapperInt.intValue();
```

6. What is autoboxing and unboxing in Java?
- Autoboxing is the automatic conversion of primitive data types to their corresponding wrapper class objects, while unboxing is the automatic conversion of wrapper class objects to their corresponding primitive data types.

7. Can you give an example of autoboxing and unboxing in Java?
- Autoboxing example:

```
Integer wrapperInt = 10; // Autoboxing
```

- Unboxing example:

```
int primitiveInt = wrapperInt; // Unboxing
```

8. What is the difference between == operator and equals() method when comparing wrapper class objects in Java?
- The == operator checks if two references point to the same memory location, while the equals() method checks if the content of the objects is equal.

9. Can you create a custom wrapper class in Java? If yes, how?
- Yes, you can create a custom wrapper class by creating a class that wraps around a primitive data type and provides methods to work with it. For example:

```
public class CustomInteger {
    private int value;

    public CustomInteger(int value) {
        this.value = value;
    }

    public int getValue() {
        return value;
```

```
    }

    // Additional methods as needed
}
```

10. How do you handle null values with wrapper classes in Java?
    - Wrapper classes in Java allow null values to be assigned, indicating the absence of a value for the primitive data type.

11. What are some common methods available in the wrapper classes in Java?
    - Common methods include intValue(), doubleValue(), floatValue(), toString(), compareTo(), equals(), and valueOf().

12. Explain the difference between Integer.parseInt() and Integer.valueOf() methods.
    - Integer.parseInt() converts a String to an int primitive, while Integer.valueOf() converts a String to an Integer object.

13. Can you explain the purpose of the Boolean.valueOf() method?
    - Boolean.valueOf() returns a Boolean object representing the Boolean value of the specified string.

14. How does the compareTo() method work in wrapper classes such as Integer and Double?
    - The compareTo() method compares two wrapper objects numerically. It returns a negative integer, zero, or a positive integer depending on whether the current object is less than, equal to, or greater than the specified object.

15. Can you explain the purpose of the Number class in Java and its relationship with wrapper classes for numeric types?
    - The Number class is an abstract superclass of wrapper classes for numeric types. It provides common functionality for these classes, such as conversion to other primitive types and methods for arithmetic operations.

Constructor

- Constructor is a special type of method that is automatically called when an instance of a class is created.
- It is used to initialize the newly created object.
- The constructor has the same name as the class and does not have a return type, not even void.
- Constructors are special methods that have the same name as the class and do not have a return type.
- Constructors are essential for initializing objects in Java, providing a way to set initial values, encapsulate initialization logic, and ensure that objects are properly initialized before they are used.

1. Initializing Object State: Constructors initialize the state of objects by setting initial values to their instance variables. This ensures that objects start in a valid and consistent state.

2. Encapsulation: Constructors are used to encapsulate the initialization logic within the class itself, making it easier to maintain and manage the object's state.

3. Creating Objects: Constructors are invoked when objects are created using the `new` keyword. They allocate memory for the object and perform any necessary setup operations.

4. Providing Default Values: Constructors can provide default values for instance variables, ensuring that objects have sensible initial values even if no explicit values are provided.

5. Constructor Overloading: Java allows constructors to be overloaded, meaning that multiple constructors can be defined in a class with different parameter lists. This provides flexibility in object creation by allowing objects to be initialized in different ways.

6. Initialization Order: Constructors help define the order in which instance variables are initialized and object construction occurs. This ensures that objects are properly initialized before they are used.

7. Automatic Invocation: Constructors are automatically invoked when objects are created, eliminating the need for manual initialization and reducing the risk of uninitialized objects.

8. Enforcing Constraints: Constructors can enforce constraints on object creation by validating input parameters and ensuring that objects are created in a valid state.

Types Of Constructor ::

1.Default Constructor: A default constructor is one that does not take any arguments. If you don't define any constructor in your class, Java automatically provides a default constructor. It initializes the instance variables to their default values (0, null, false, etc.) if they are not explicitly initialized.

```
public class MyClass {
    // Default constructor provided by Java if not explicitly defined
}
```

2. Parameterized Constructor: A parameterized constructor is one that takes parameters to initialize the instance variables of the class.

```
public class Person {
    private String name;
    private int age;

    // Parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

3. Copy Constructor: A copy constructor creates a new object by copying the state of an existing object. It takes an object of the same class as a parameter and initializes the new object with the values from the existing object.

```
public class Point {
    private int x;
    private int y;

    // Copy constructor
    public Point(Point other) {
        this.x = other.x;
        this.y = other.y;
    }
}
```

4. Constructor Overloading: Constructor overloading is when a class has multiple constructors, each with a different parameter list. This allows objects to be initialized in different ways.

```
public class Rectangle {
    private int length;
    private int width;

    // Parameterized constructor 1
    public Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }

    // Parameterized constructor 2 (overloaded)
    public Rectangle(int side) {
        this.length = side;
        this.width = side;
    }
}
```

5. Private Constructor: A private constructor is one that is not accessible from outside the class. It is often used in classes that contain only static methods and cannot be instantiated.

```
public class UtilityClass {
    // Private constructor to prevent instantiation
    private UtilityClass() {
        // This constructor is private, so the class cannot be instantiated
```

```
    }

    public static void doSomething() {
        // Some static method
    }
}
```

****FAQs (Frequently Asked Questions) for Constructors in Java:****

1. What is a constructor in Java?
   - A constructor in Java is a special type of method that is automatically invoked when an object of a class is created. It initializes the newly created object.

2. What is the purpose of a constructor?
   - The main purpose of a constructor is to initialize the state of an object, typically by initializing its member variables.

3. What are the rules for defining a constructor?
   - The constructor must have the same name as the class.
   - It does not have a return type, not even void.
   - It can have parameters.
   - It can be overloaded.
   - If no constructor is explicitly defined, Java provides a default constructor with no arguments.

4. Can a constructor be inherited?
   - Constructors are not inherited in Java. However, a subclass can invoke a constructor of the superclass using the `super()` keyword.

5. Can a constructor be private or protected?
   - Yes, a constructor can be private or protected. Private constructors are typically used in utility classes to prevent instantiation, while protected constructors are used in inheritance scenarios.

6. Can a constructor return a value?
   - No, a constructor cannot return a value, not even void. Its purpose is solely to initialize the object.

7. Can constructors be used to initialize static variables?
   - No, constructors are used to initialize instance variables. Static variables are initialized using static initialization blocks or static initializers.

8. Can we have multiple constructors in a class?
   - Yes, a class can have multiple constructors with different parameter lists. This is called constructor overloading.

9. What is constructor chaining?
   - Constructor chaining is the process of calling one constructor from another constructor within the same class or from the subclass constructor using the `this()` or `super()` keyword.

10. When is the default constructor invoked?

- The default constructor is invoked when no other constructor is explicitly defined in the class and an object of that class is created without arguments.

11**. What is a copy constructor**?

- A copy constructor is a constructor that creates a new object by copying the state of an existing object. It is typically used to create a deep copy of an object.

12. How do you prevent a class from being instantiated using constructors?

- You can make the constructor private to prevent instantiation of the class from outside. This is commonly used in utility classes where instances are not desired.

Generics

- Generics in Java provide a way to create classes, interfaces, and methods that operate on specified types. They allow you to parameterize types so that you can use them with any data type.
- Generics are used in Java to achieve type safety, code reusability, and improved readability. They allow developers to write flexible and reusable code that can work with different types without sacrificing type safety.
- Generics allows us to write code that can operate on a variety of data types without sacrificing type safety or code duplication.

Problem

- Before the introduction of generic classes in Java, developers often encountered the problem of type safety and code duplication when working with collections and data structures.
- ClassCastExceptions at Runtime.

Solution

- Generic classes in Java provide a solution to these problems by allowing developers to define classes and methods with type parameters.
- Generics in Java help prevent ClassCastExceptions by providing compile-time type safety. This helps catch type mismatches at compile time rather than waiting for runtime, where they would result in ClassCastExceptions.

Types

1. Bounded Generics:

- These restrict the types that can be used as type parameters.
- There are two kinds of bounded generics.

Upper Bounded:

- Specified using the extends keyword. It restricts the type to be a specific type or any of its subclasses.

  class MyClass<T extends Number> { /* ... */ }

Lower Bounded:

- Specified using the super keyword. It restricts the type to be a specific type or any of its superclasses.

  class MyClass<T super Number> { /* ... */ }

2. Unbounded Generics:

- These are generic types without any restriction. You can use any type as the type parameter.

  class MyClass<T> { /* ... */ }

3.Wildcards:

- Wildcards allow you to create more flexible generic classes, methods, or interfaces by using ? symbol.

There are three types of wildcards:

1. **Upper Bounded Wildcard:**
   - It represents all the classes that are children of the specified class.

     public void myMethod(List<? extends Number> list) { /* ... */ }
2. **Lower Bounded Wildcard:**
   - It represents all the classes that are parents of the specified class.

     public void myMethod(List<? super Integer> list) { /* ... */ }
3. **Unbounded Wildcard:**
   - It represents all types

     public void myMethod(List<?> list) { /* ... */ }

Type Erasure

- Generics were added to Java to ensure type safety. And to ensure that generics won't cause overhead at runtime, the compiler applies a process called type erasure on generics at compile time.
- Type erasure removes all type parameters and replaces them with their bounds or with Object if the type parameter is unbounded. This way, the bytecode after compilation contains only normal classes, interfaces and methods, ensuring that no new types are produced. Proper casting is applied as well to the Object type at compile time.

Raw Types

- A raw type is a name for a generic interface or class without its type argument.

  List list = new ArrayList(); // raw type

Interview Questions

---

1. What are generics in Java?

Generics in Java provide a way to create classes, interfaces, and methods that operate on specified types. They allow you to parameterize types so that you can use them with any data type.

2. Why are generics used in Java?

Generics are used in Java to achieve type safety, code reusability, and improved readability. They allow developers to write flexible and reusable code that can work with different types without sacrificing type safety.

3. Explain the concept of type erasure in Java generics.

Type erasure is the process by which the compiler removes generic type information from the compiled bytecode. This is done to maintain compatibility with older Java code that does not support generics at runtime.

4. What is a type parameter in Java generics?

A type parameter in Java generics is a placeholder for a type that will be specified when the generic class, interface, or method is used. It allows you to create classes, interfaces, and methods that can work with any reference type.

5. How do you declare a generic class in Java?

To declare a generic class in Java, you use angle brackets (`<>`) to specify one or more type parameters after the class name. For example: `public class Box<T> { ... }`

6. What is the purpose of wildcard types in Java generics?

Wildcard types in Java generics are used to represent unknown types or to define generic methods that operate on any type. They provide flexibility when working with generic types by allowing you to accept any type without specifying the exact type.

7. What is the difference between `List<T>` and `List<?>` in Java generics?

`List<T>` represents a list of elements of type `T`, while `List<?>` represents a list of elements of an unknown type. The former is more restrictive, as it specifies a specific type, while the latter accepts any type.

8. Explain the difference between upper bounded wildcards (`? extends T`) and lower bounded wildcards (`? super T`).

Upper bounded wildcards (`? extends T`) restrict the wildcard to be a subtype of `T`, while lower bounded wildcards (`? super T`) restrict the wildcard to be a supertype of `T`.

9. What is the purpose of the `<? extends Number>` wildcard in Java generics?

The `<? extends Number>` wildcard represents any type that is a subtype of `Number`. It allows you to accept any subclass of `Number`, such as `Integer`, `Double`, or `Float`.

10. How can you restrict the types that can be used as type arguments in a generic class or method?

You can restrict the types by using bounded type parameters. For example, `public class Box<T extends Number> { ... }` restricts `T` to be a subtype of `Number`.

11. Can you create an array of a generic type in Java? Why or why not?

No, you cannot create an array of a generic type directly in Java due to type erasure. However, you can create an array of a generic type by using a raw type or casting.

12. What is type inference in Java generics?

Type inference in Java generics allows the compiler to automatically determine the type arguments of a generic method or constructor based on the context in which it is called.

13. How do you handle generic types with inheritance in Java?

You can use bounded wildcards to handle generic types with inheritance in Java. This allows you to specify upper or lower bounds on the wildcard to accept subclasses or superclasses.

14. Explain the concept of raw types in Java generics.

Raw types in Java generics are generic types that are used without specifying any type parameters. They are compatible with legacy code that does not support generics and bypass compile-time type checks.

15. What is the difference between bounded and unbounded wildcards in Java generics?

Bounded wildcards (`? extends T` or `? super T`) impose restrictions on the wildcard, while unbounded wildcards (`?`) accept any type.

16. Can you give an example of a situation where you would use a wildcard type?

An example situation where you would use a wildcard type is when writing a method that accepts a collection of unknown types, such as `public void printList(List<?> list) { ... }`.

17. How do you ensure type safety when using generics in Java?

Type safety in generics is ensured by specifying appropriate type parameters and using bounded wildcards when necessary. Additionally, thorough testing and code reviews can help ensure type safety.

18. Explain how generics improve code readability and maintainability.

Generics improve code readability and maintainability by making code more expressive and reducing the need for explicit type casting. They also promote code reuse and allow for more flexible and generic implementations.

19. What are the benefits of using generics compared to using raw types?

The benefits of using generics over raw types include improved type safety, better code readability, and enhanced compile-time error checking. Generics also provide better documentation and enable better IDE support.

20. How does Java generics compare to generics in other programming languages?

Java generics are similar to generics in other programming languages like C# and Kotlin but have some differences in syntax and implementation. Java's type erasure approach differs from other languages that may use reification or other methods to retain type information at runtime.

Five different ways to create objects in Java

---

1) **Using the `new` Keyword:**

   This is the most common way to create objects in Java.

   Syntax: `ClassName objectName = new ClassName();`

2) **Using `newInstance()` Method:**

This method is part of the `Class` class and is used to create new instances of classes dynamically at runtime.

Syntax:

```
ClassName obj = (ClassName) Class.forName("ClassName").newInstance();
// or
ClassName obj = ClassName.class.newInstance();
```

3) **Using `Constructor.newInstance()` Method:**

   This method allows creating objects by invoking the constructor of a class explicitly.

   Syntax:

```
Constructor<ClassName> constructor = ClassName.class.getConstructor();
ClassName obj = constructor.newInstance();
```

4) Using `clone()` Method:

- This method is used to create a copy of an existing object.
- The class whose objects need to be cloned must implement the `Cloneable` interface and override the `clone()` method.

Syntax:
ClassName obj = (ClassName) existingObj.clone();

5) Using Serialization/Deserialization:

- Serialization is the process of converting an object into a byte stream, which can be stored in a file or sent over a network.
- Deserialization is the process of reconstructing the object from the serialized byte stream.

Syntax:

```
// Serialization
ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("filename.ser"));
out.writeObject(obj);
out.close();

// Deserialization
ObjectInputStream in = new ObjectInputStream(new FileInputStream("filename.ser"));
ClassName obj = (ClassName) in.readObject();
in.close();
```

Java is pass/call by value or pass/call by reference

- By default java is pass by Value .
- Java is strictly pass-by-value. This means that when you pass arguments to a method, a copy of the value of each argument is passed to the method, rather than the actual object itself.

Pass By value

```
public class PassByValueExample {
   public static void main(String[] args) {
      int num = 10;
      System.out.println("Before calling method: " + num);
      modifyValue(num);
      System.out.println("After calling method: " + num);
   }
   public static void modifyValue(int num) {
      num = 20; // Modify the local copy of num
      System.out.println("Inside method: " + num);
   }
}
```

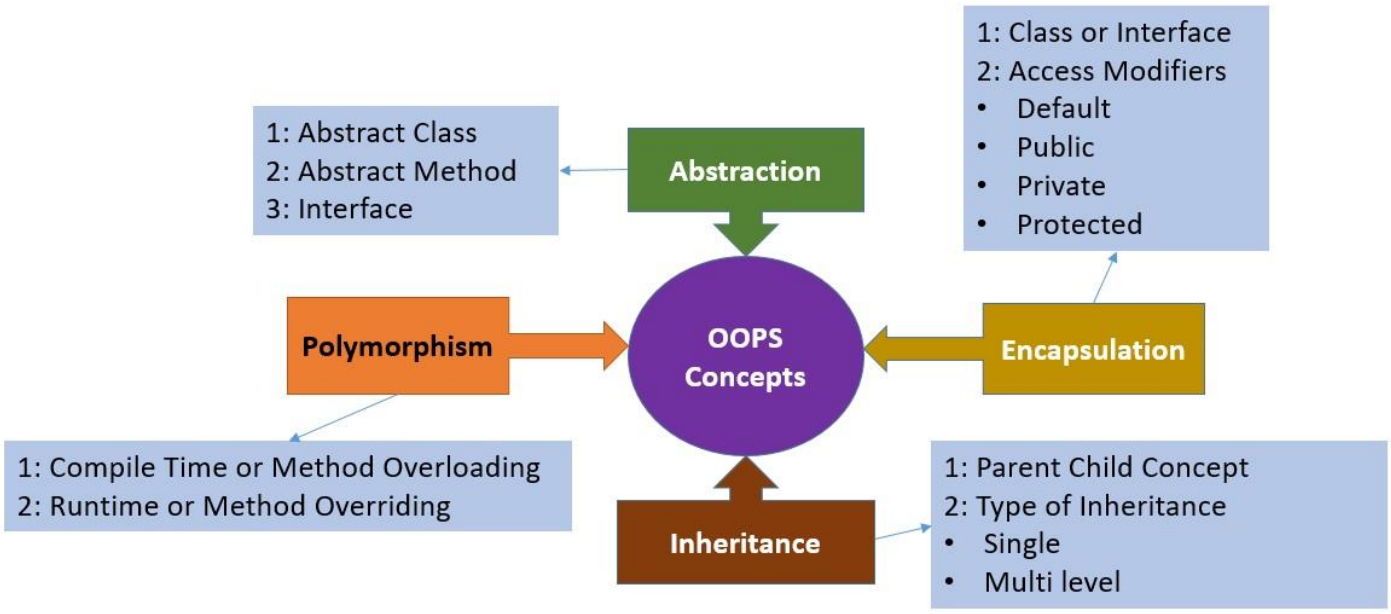
Pass By reference

```
public class PassByValueExample {
   public static void main(String[] args) {
      StringBuilder str = new StringBuilder("Hello");
      System.out.println("Before calling method: " + str);
      modifyStringBuilder(str);
      System.out.println("After calling method: " + str);
```

```
    }

    public static void modifyStringBuilder(StringBuilder str) {
        str.append(" World"); // Modify the object referred to by str
        System.out.println("Inside method: " + str);
    }
}
```

OOPS



OOPS Concepts

1. What is Object-Oriented Programming (OOP)?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data in the form of fields (attributes or properties) and code in the form of procedures (methods or functions). OOP focuses on the organization of code into reusable components and emphasizes concepts such as encapsulation, inheritance, polymorphism, and abstraction.

2. What are the four pillars of Object-Oriented Programming?

The four pillars of OOP are:

- Encapsulation: Bundling of data and methods that operate on the data into a single unit.

- Inheritance: Mechanism by which a class can inherit properties and behavior from another class.

- Polymorphism: Ability of objects of different types to be treated as objects of a common superclass.

- Abstraction: Process of hiding the complex implementation details and showing only essential features of an object.

3. Explain Encapsulation in Java.

Encapsulation is the bundling of data (fields) and methods that operate on the data into a single unit (class). It allows for the hiding of the internal state of objects and provides controlled access to the data through methods. This helps in achieving data abstraction and protects the integrity of data by preventing unauthorized access.

4. What is Inheritance in Java?

Inheritance is a mechanism in Java by which one class (subclass) can inherit properties and behavior from another class (superclass). It facilitates code reuse and establishes hierarchical relationships between classes. The subclass can inherit fields and methods from the superclass and can also define its own fields and methods.

5. Explain Polymorphism in Java.

Polymorphism is the ability of objects of different types to be treated as objects of a common superclass. It allows methods to be invoked dynamically based on the actual type of the object at runtime. Polymorphism is achieved in Java through method overriding and method overloading.

6. What is Abstraction in Java?

Abstraction is the process of hiding the complex implementation details and showing only essential features of an object. It focuses on what an object does rather than how it does it. Abstraction helps in reducing complexity, managing dependencies, and improving code maintainability.

7. What is the difference between method overloading and method overriding?

Method overloading involves defining multiple methods in the same class with the same name but different parameters. Method overriding occurs when a subclass provides a specific implementation of a method that is already provided by its superclass. Method overloading is resolved at compile time based on the method signature, while method overriding is resolved at runtime based on the actual type of the object.

8. What is the `super` keyword in Java?

In Java, the `super` keyword is used to refer to the superclass of the current object instance. It is primarily used to access methods, constructors, and instance variables of the superclass from within the subclass. `super()` is used to call the superclass constructor from the subclass constructor.

9. Explain the concept of interface in Java.

An interface in Java is a reference type similar to a class but can only contain constants, method signatures, default methods, static methods, and nested types. It defines a contract for classes to implement, specifying a set of methods that the implementing class must provide. Interfaces are used to achieve abstraction, multiple inheritance, and loose coupling.

10. What is the difference between abstract class and interface in Java?

An abstract class in Java can contain both abstract methods (methods without a body) and concrete methods, whereas an interface can only contain abstract methods, default methods, and static methods. A class can implement multiple interfaces but can extend only one abstract class. Abstract classes can have constructors, whereas interfaces cannot.

11. What are access modifiers in Java? Explain the different types.

Access modifiers in Java are keywords that specify the accessibility or scope of classes, methods, and variables. The different types of access modifiers are:

- `public`: Accessible from any other class.
- `protected`: Accessible within the same package and by subclasses.
- `default` (no modifier): Accessible within the same package.
- `private`: Accessible only within the same class.

12. What is method hiding in Java?

Method hiding occurs when a subclass defines a static method with the same signature as a static method in its superclass. Unlike method overriding, where the method to be invoked is determined at runtime based on the actual object type, method hiding involves choosing the method to be invoked based on the reference type.

13. Explain the concept of constructor chaining in Java.

Constructor chaining refers to the process of calling one constructor from another constructor in the same class or in the superclass. This is achieved using the `this()` keyword to call another constructor in the same class or the `super()` keyword to call a constructor in the superclass. Constructor chaining allows for code reuse and ensures that common initialization logic is centralized.

14. What is composition in Java? Provide an example.

Composition is a design principle where a class contains references to other classes as member variables. It allows objects to be composed of other objects, enabling code reuse, modularity, and maintainability. For example, a `Car` class may contain instances of `Engine`, `Wheel`, and `Body` classes as its member variables.

15. What is the `instanceof` operator in Java? How is it used?

The `instanceof` operator in Java is used to check whether an object is an instance of a particular class or interface. It returns `true` if the object is an instance of the specified class or implements the specified interface; otherwise, it returns `false`. It's often used in conditional statements and type casting to ensure safe object manipulation.

16. What is method visibility or method scope in Java?

Method visibility or method scope in Java refers to the accessibility of a method from other classes. The visibility of a method is determined by its access modifier. Methods with the `public` access modifier are accessible from any other class, methods with the `protected` access modifier are accessible within the same package and by subclasses, methods with the default (no modifier) access modifier are accessible within the same package, and methods with the `private` access modifier are accessible only within the same class.

17. Explain method overriding and access modifiers.

Method overriding occurs when a subclass provides a specific implementation of a method that is already provided by its superclass. Access modifiers control the visibility of methods in Java. When overriding a method in a subclass, the access modifier of the overridden method in the superclass can be the same or less restrictive, but not more restrictive.

18. What is method visibility or method scope in Java?

Method visibility or method scope in Java refers to the accessibility of a method from other classes. The visibility of a method is determined by its access modifier. Methods with the `public` access modifier are accessible from any other class, methods with the `protected` access modifier are accessible within the same package and by subclasses, methods with the default (no modifier) access modifier are accessible within the same package, and methods with the `private` access modifier are accessible only within the same class.

19. Explain method hiding in Java.

Method hiding occurs when a subclass defines a static method with the same signature as a static method in its superclass. Unlike method overriding, where the method to be invoked is determined at runtime based on the actual object type, method hiding involves choosing the method to be invoked based on the reference type.

20. What is the purpose of the `super` keyword in Java?

In Java, the `super` keyword is used to refer to the superclass of the current object instance. It is primarily used to access methods, constructors, and instance variables of the superclass from within the subclass. `super()` is used to call the superclass constructor from the subclass constructor.

Exception handling

Exceptions::

- An unwanted unexpected event that disturbs normal flow of the program is called exception .
- It is highly recommended to handle exceptions. The main objective of exception handling is graceful (normal) termination of the program.
    - Ex : If **FileNotFoundException** occurs then we can use local file and we can continue rest of the program execution normally.

Types of Exception

Checked:

The exceptions which are checked by the compiler whether programmer handling or not, for smooth execution of the program at runtime, are called checked exceptions.

Ex:FileNotFoundException

Unchecked Exceptions :

The exceptions which are not checked by the compiler whether programmer handing or not ,are called unchecked exceptions.

Ex:NullPointerException

Note:RuntimeException and its child classes, Error and its child classes are unchecked and all the remaining are considered as checked exceptions.

Errors ::

- Error is a type of Throwable that represents serious problems that are typically beyond the control of the application.
- Errors are often associated with problems at the system level or issues that are too severe to recover from.
- Errors are not meant to be caught or handled by typical application code.
    - **OutOfMemoryError**: This error occurs when the Java Virtual Machine (JVM) cannot allocate enough memory to fulfill an allocation request, typically due to the heap being exhausted.
    - **StackOverflowError**: This error occurs when the stack space allocated to a thread is exhausted, usually due to infinite recursion or excessively deep function calls.
    - **AssertionError**: This error is thrown to indicate that an assertion has failed. Assertions are typically used to check for conditions that should always be true and indicate programming errors if they fail.

Default Exception Handling in Java:

1. If an exception raised inside any method then that method is responsible to create Exception object with the following information.

1. Name of the exception.
2. Description of the exception.
3. Location of the exception.(StackTrace)

2. After creating that Exception object, the method handovers that object to the JVM.

3. JVM checks whether the method contains any exception handling code or not. If method won't contain any handling code then JVM terminates that method abnormally and removes corresponding entry form the stack.

4. JVM identifies the caller method and checks whether the caller method contain any handling code or not. If the caller method also does not contain handling code then JVM terminates that caller method also abnormally and removes corresponding entry from the stack.

5. This process will be continued until main() method and if the main() method also doesn't contain any exception handling code then JVM terminates main() method also and removes corresponding entry from the stack.

6. Then JVM handovers the responsibility of exception handling to the default exception handler.

Why we are Using ::

try - To maintain Reisky Code

catch - To maintain Exception Handle Code.

finally - To maintain cleanup activity.

throw - To hand-over our created exception object to the JVM manually.

throws - To delegate responsibility of exception handling t caller .

The difference between "throw" and "throws" in Java is:

throw: It is used to manually throw an exception within a method. When you use "throw", you're indicating that an exceptional condition has occurred and you want to create and throw an exception object manually at a specific point in your code.

Ex::

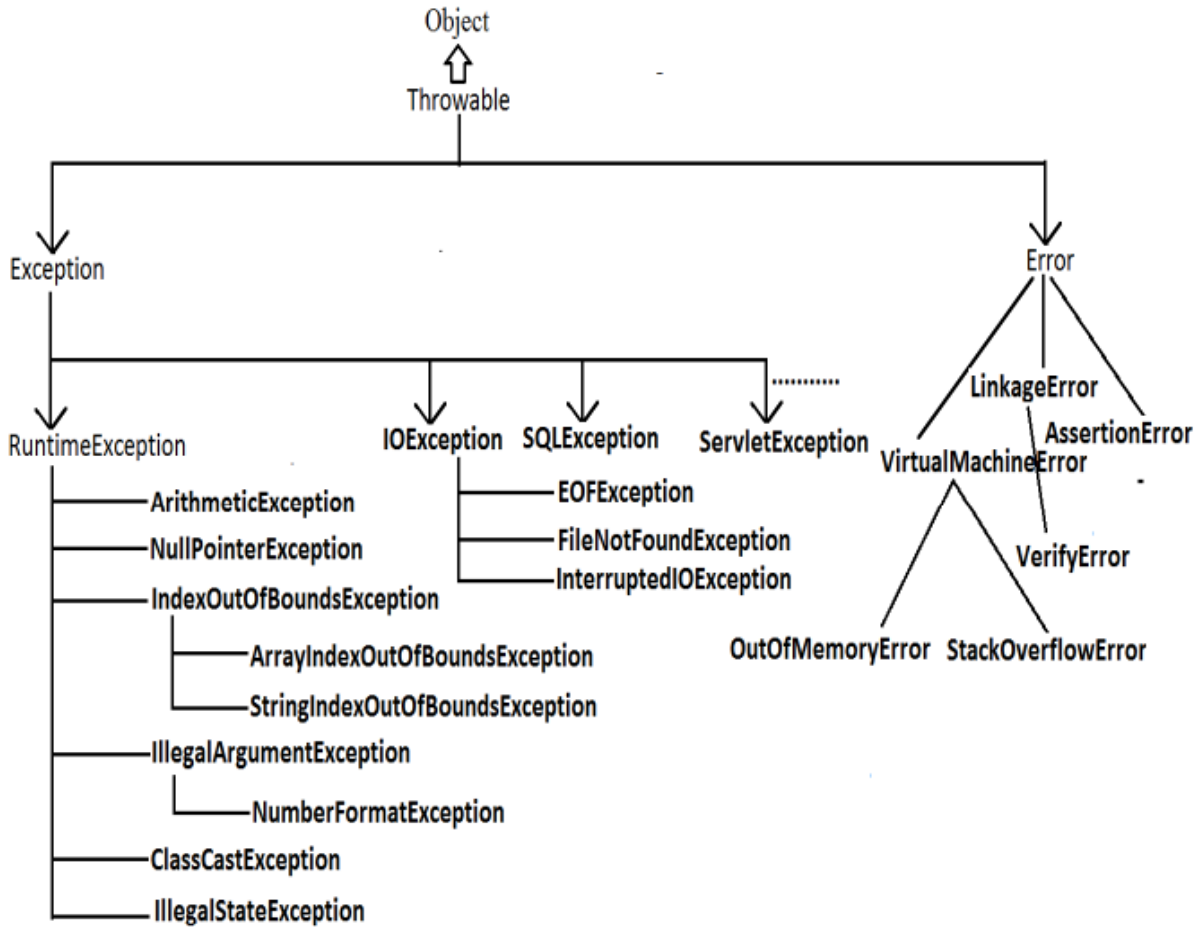
```
public void checkAge(int age) {
   if (age < 0) {
      throw new IllegalArgumentException("Age must be non-negative");
   }
}
```

throws: It is used in method declarations to indicate that the method may throw certain types of exceptions. When you use "throws", you're declaring that the method might throw certain exceptions, allowing callers of the method to anticipate and handle these exceptions or propagate them further.

Ex::

```
public void readFile(String filename) throws IOException {
   // Code to read a file
}
```

Exception Hierrarchy ::



Exception handling Interview Questions::

---

1. Explain the difference between checked and unchecked exceptions in Java.

2. Can you catch multiple exceptions in a single catch block? If yes, how?

3. What is the purpose of the "finally" block in exception handling?

4. Is it possible to have a "finally" block without a corresponding "try" block?

5. How do you handle exceptions in multithreaded environments?

Exceptions in multithreaded environments can be handled using constructs like try-catch blocks within each thread or using uncaught exception handlers to manage exceptions that propagate out of threads.

6. What is the significance of the "throws" clause in method declarations?

The "throws" clause in method declarations is used to indicate that a method may throw certain types of exceptions. It specifies the exceptions that can be thrown by the method but does not handle them.

7. Explain the difference between "throw" and "throws" in Java.

"throw" is used to explicitly throw an exception within a method, while "throws" is used in method declarations to indicate the exceptions that may be thrown by the method .

8. Can a method throw multiple exceptions? If so, how do you handle them?

Yes, a method can throw multiple exceptions. Each exception can be handled separately using multiple catch blocks or caught using a common superclass of the exceptions.

9. What happens if an exception is thrown from within a "finally" block?

If an exception is thrown from within a "finally" block, it will override any previous exception thrown in the try or catch blocks. The new exception will propagate up the call stack .

10. How does Java handle stack overflow errors?

Stack overflow errors are typically caused by infinite recursion or excessively deep function calls. Java handles stack overflow errors by throwing a "StackOverflowError" runtime exception.

11. What is the purpose of the "try-with-resources" statement in Java?

The "try-with-resources" statement in Java ensures that resources are closed properly after being used, even if an exception occurs. It automatically closes resources that implement the "AutoCloseable" interface .

12. How can you customize exception handling in Java?

13. Is it possible to recover from an exception in Java? If yes, how?

In some cases, it's possible to recover from exceptions in Java by catching and handling them appropriately. For example, by retrying the operation or providing alternative paths of execution.

14. Can you explain the concept of "chained exceptions"?

Chained exceptions allow you to associate one exception with another.

16. Explain the difference between the "Error" and "Exception" classes in Java.

"Error" and "Exception" are both subclasses of "Throwable" in Java. "Error" represents serious problems that typically cannot be handled by the application, while "Exception" represents exceptional conditions that can be caught and handled.

17. Can you create your custom exception class in Java? If yes, when would you do so?

Yes, you can create custom exception classes in Java by extending either the "Exception" class for checked exceptions or the "RuntimeException" class for unchecked exceptions.

18. How do you ensure resource cleanup in case of exceptions in Java?

Resource cleanup in case of exceptions can be ensured by using the "try-with-resources" statement or by implementing proper exception handling logic to close resources in the "finally" block.

19. Can you nest try-catch blocks in Java? If yes, is there any limit?

20. How do you handle exceptions gracefully in a web application?

Exceptions in web applications can be handled using frameworks like Spring MVC, which provide mechanisms for centralized exception handling, custom error pages, and logging.

21. ClassCastException vs NoClassDefFoundError

NoClassDefFoundError:

- This error occurs when the Java Virtual Machine (JVM) or a class loader tries to load a class definition but cannot find the appropriate .class file at runtime.
- It typically happens when a class that was available during compile time is missing during runtime.

ClassCastException:

- This exception occurs at runtime when you try to cast an object to a type that is not compatible with its actual type.
- It typically happens in situations where you have polymorphic code and you attempt to cast an object to a subclass type that it is not actually an instance of.

MultiThreading

Multitasking: Executing several tasks simultaneously is the concept of multitasking.

There are two types of multitasking's.

1. Process based multitasking.
2. Thread based multitasking.

Process based multitasking

- Executing several tasks simultaneously where **each task is a separate independent process** such type of multitasking is called process based multitasking.

Example:

- While typing a java program in the editor we can able to listen mp3 audio songs at the same time we can download a file from the net all these tasks are independent of each other and executing simultaneously and hence it is Process based multitasking.
- This type of multitasking is best suitable at "os level".

Thread based multitasking:

- Executing several tasks simultaneously where **each task is a separate independent part of the same program**, is called Thread based multitasking.
- And each independent part is called a "Thread".

❖ The ways to define instantiate and start a new Thread

1. By extending Thread class.
2. By implementing Runnable interface

Case 1: Thread Scheduler:

- If multiple Threads are waiting to execute then which Thread will execute 1st is decided by "Thread Scheduler" which is part of JVM.
- Which algorithm or behavior followed by Thread Scheduler we can't expect exactly it is the JVM vendor dependent hence in multithreading examples we can't expect exact execution order and exact output .

Case 2: Difference between t.start() and t.run() methods.

- In the case of t.start() a new Thread will be created which is responsible for the execution of run() method.
- But in the case of t.run() no new Thread will be created and run() method will be executed just like a normal method by the main Thread.

Case 3: importance of Thread class start() method.

- For every Thread the required mandatory activities like registering the Thread with Thread Scheduler will takes care by Thread class start() method and programmer is responsible just to define the job of the Thread inside run() method.
- That is start() method acts as best assistant to the programmer.

Example:

```
start()
{
1. Register Thread with Thread Scheduler
2. All other mandatory low level activities.
3. Invoke or calling run() method.
}
```

- We can conclude that without executing Thread class start() method there is no chance of starting a new Thread in java. Due to this start() is considered as heart of multithreading.

Case 4: If we are not overriding run() method:

- If we are not overriding run() method then Thread class run() method will be executed which has empty implementation and hence we won't get any output.
- It is highly recommended to override run() method. Otherwise don't go for multithreading concept.

Case 5: Overloding of run() method.

- We can overload run() method but Thread class start() method always invokes no argument run() method the other overload run() methods we have to call explicitly then only it will be executed just like normal method.

Case 6: overriding of start() method:

- If we override start() method then our start() method will be executed just like a normal method call and no new Thread will be started.

Best approach to define a Thread:

- Among the 2 ways of defining a Thread, **implements Runnable approach** is always recommended.
- In the 1st approach our class should always extends Thread class there is no chance of extending any other class hence we are missing the benefits of inheritance.
- But in the 2nd approach while implementing Runnable interface we can extend some other class also. Hence implements Runnable mechanism is recommended to define a Thread.

Getting and setting name of a Thread:

- Every Thread in java has some name it may be provided explicitly by the programmer or automatically generated by JVM.
- Thread class defines the following methods to get and set name of a Thread.

Methods:

1. public final String getName()
2. public final void setName(String name)

Note: We can get current executing Thread object reference by using Thread.currentThread() method.

Thread Priorities

- Every Thread in java has some priority it may be default priority generated by JVM (or) explicitly provided by the programmer.
- The valid range of Thread priorities is 1 to 10[but not 0 to 10] where **1 is the least priority and 10 is highest priority**.
- Thread class defines the following constants to represent some standard priorities.
    1. Thread. MIN_PRIORITY----------1
    2. Thread. MAX_PRIORITY----------10
    3. Thread. NORM_PRIORITY--------5
- There are no constants like Thread.LOW_PRIORITY,Thread.HIGH_PRIORITY
- Thread scheduler uses these priorities while allocating CPU.
- The Thread which is having highest priority will get chance for 1st execution.
- If 2 Threads having the same priority then we can't expect exact execution order it depends on Thread scheduler whose behavior is vendor dependent.
- We can get and set the priority of a Thread by using the following methods.
    1. public final int **getPriority**()
    2. public final void **setPriority**(int newPriority);//the allowed values are 1 to 10
- The allowed values are 1 to 10 otherwise we will get runtime exception saying "IllegalArgumentException".

Default priority:

The default priority only for the main Thread is **5**. But for all the remaining Threads the default priority will be inheriting from parent to child. That is whatever the priority parent has by default the same priority will be for the child also.

The Methods to Prevent a Thread from Execution:

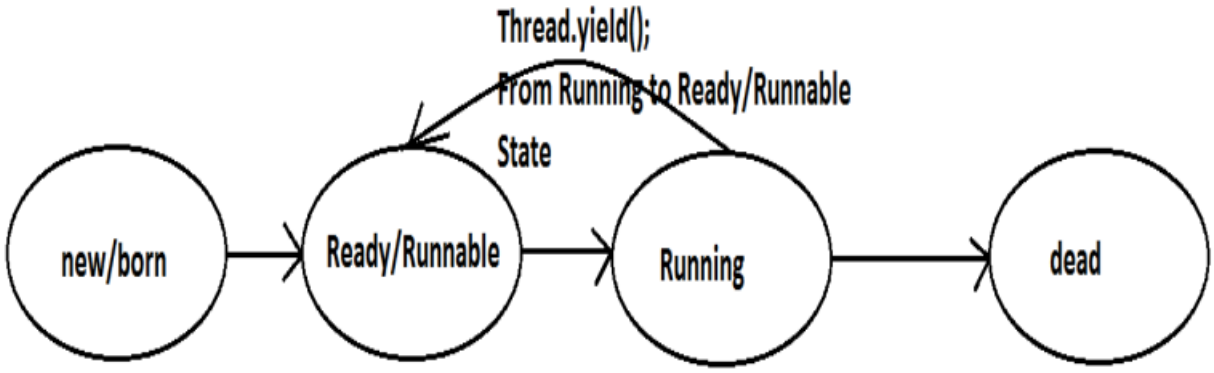
We can prevent(stop) a Thread execution by using the following methods.

1. **yield();**
2. **join();**
3. **sleep();**

yield():

1. yield() method causes "to pause current executing Thread for giving the chance of remaining waiting Threads of same priority".

2. If all waiting Threads have the low priority or if there is no waiting Threads then the same Thread will be continued its execution.
3. If several waiting Threads with same priority available then we can't expect exact which Thread will get chance for execution.
4. The Thread which is yielded when it get chance once again for execution is depends on mercy of the Thread scheduler.
5. public static native void yield();



Syntax:: Thread.yield();

Join():

- If a Thread wants to wait until completing some other Thread then we should go for join() method.
  Example: If a Thread t1 executes t2.join() then t1 should go for waiting state until completing t2.
  1. public final void join()throws InterruptedException
  2. public final void join(long ms) throws InterruptedException
  3. public final void join(long ms,int ns) throws InterruptedException
- Every join() method throws InterruptedException, which is checked exception hence compulsory we should handle either by try catch or by throws keyword. Otherwise we will get compiletime error.

  Syntax:: thread.join();

Sleep() method:

- If a Thread don't want to perform any operation for a particular amount of time then we should go for sleep() method.
  1. public static native void sleep(long ms) throws InterruptedException
  2. public static void sleep(long ms,int ns)throws InterruptedException

Interupt()

If thread is on Waiting state or sleep state then only we can interrupt that thread.

Thread.interput();

Synchronization

1. Synchronized is the keyword applicable for methods and blocks but not for classes and variables.
2. If a method or block declared as the synchronized then at a time only one Thread is allow to execute that method or block on the given object.
3. The main advantage of synchronized keyword is we can resolve date inconsistency problems.
4. But the main disadvantage of synchronized keyword is it increases waiting time of the Thread and effects performance of the system.
5. Hence if there is no specific requirement then never recommended to use synchronized keyword.
6. Internally synchronization concept is implemented by using lock concept.
7. Every object in java has a unique lock. Whenever we are using synchronized keyword then only lock concept will come into the picture.

8. If a Thread wants to execute any synchronized method on the given object 1st it has to get the lock of that object. Once a Thread got the lock of that object then it's allow to execute any synchronized method on that object. If the synchronized method execution completes then automatically Thread releases lock.
9. While a Thread executing any synchronized method the remaining Threads are not allowed execute any synchronized method on that object simultaneously. But remaining Threads are allowed to execute any non-synchronized method simultaneously.
10. [lock concept is implemented based on **object** but not based on **method**].

Class level lock:

1. Every class in java has a unique lock. If a Thread wants to execute a static synchronized method then it required class level lock.
2. Once a Thread got class level lock then it is allow to execute any static synchronized method of that class.
3. While a Thread executing any static synchronized method the remaining Threads are not allow to execute any static synchronized method of that class simultaneously.
4. But remaining Threads are allowed to execute normal synchronized methods,normal static methods, and normal instance methods simultaneously.
5. Class level lock and object lock both are different and there is no relationship between these two.

Synchronized block:

1. If very few lines of the code required synchronization then it's never recommended to declare entire method as synchronized we have to enclose those few lines of the code with in synchronized block.
2. The main advantage of synchronized block over synchronized method is it reduces waiting time of Thread and improves performance of the system.

Inter Thread communication (wait(),notify(), notifyAll()):

- Two Threads can communicate with each other by using wait(), notify() and notifyAll() methods.
- The Thread which is required updation it has to call wait() method on the required object then immediately the Thread will entered into waiting state.
- The Thread which is performing updation of object, it is responsible to give notification by calling notify() method.
- After getting notification the waiting Thread will get those updations wait(), notify() and notifyAll() methods are available in Object class but not in Thread class because Thread can call these methods on any common object.
- To call wait(), notify() and notifyAll() methods compulsory the current Thread should be owner of that object
  i.e., current Thread should has lock of that object
  i.e., current Thread should be in synchronized area. Hence we can call wait(), notify() and notifyAll() methods only from synchronized area otherwise we will get runtime exception saying IllegalMonitorStateException.
- Once a Thread calls wait() method on the given object 1st it releases the lock of that object immediately and entered into waiting state.
- Once a Thread calls notify() (or) notifyAll() methods it releases the lock of that object but may not immediately.
- Except these (wait(),notify(),notifyAll()) methods there is no other place(method) where the lock release will be happen.
- Once a Thread calls wait(), notify(), notifyAll() methods on any object then it releases the lock of that particular object but not all locks it has.
  1. public final void wait()throws InterruptedException
  2. public final native void wait(long ms)throws InterruptedException
  3. public final void wait(long ms,int ns)throws InterruptedException
  4. public final native void notify()
  5. public final void notifyAll()

Dead lock:

- If 2 Threads are waiting for each other forever(without end) such type of situation(infinite waiting) is called dead lock.
- There are no resolution techniques for dead lock but several prevention(avoidance) techniques are possible.
- Synchronized keyword is the cause for deadlock hence whenever we are using synchronized keyword we have to take special care .

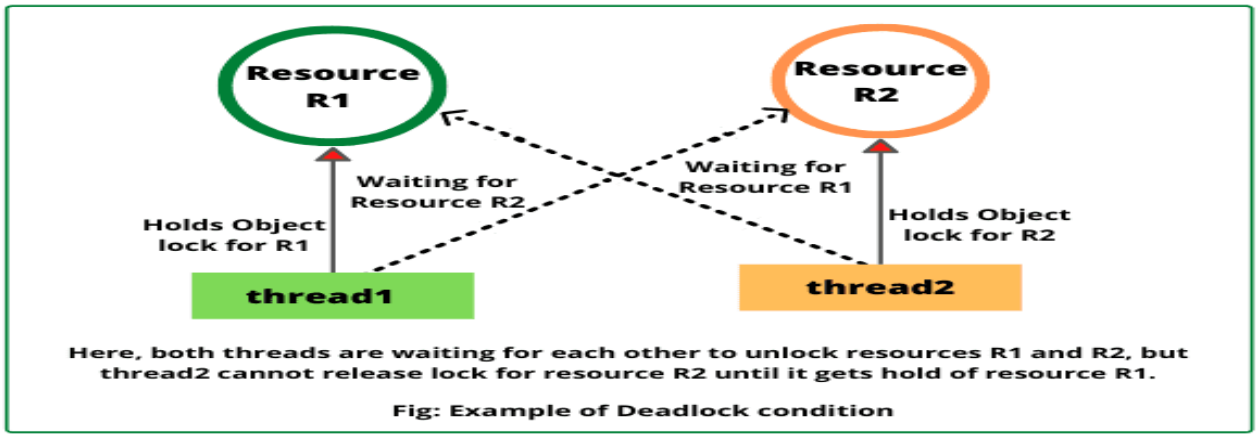
Daemon Threads:

- The Threads which are executing in the background are called daemon Threads.
- The main objective of daemon Threads is to provide support for non-daemon Threads like main Thread.

Example: Garbage collector

Deadlock vs Starvation:

- A long waiting of a Thread which never ends is called deadlock.
- A long waiting of a Thread which ends at certain point is called **starvation**.
- A low priority Thread has to wait until completing all high priority Threads.
- This long waiting of Thread which ends at certain point is called starvation.



Here, both threads are waiting for each other to unlock resources R1 and R2, but thread2 cannot release lock for resource R2 until it gets hold of resource R1.

Fig: Example of Deadlock condition

RACE condition:

- Executing multiple Threads simultaneously and causing data inconsistency problems is nothing but **Race condition.**
- We can resolve race condition by using synchronized keyword.

GreenThread:

- Java multiThreading concept is implementing by using the following 2 methods :
    1. GreenThread Model
    2. Native OS Model

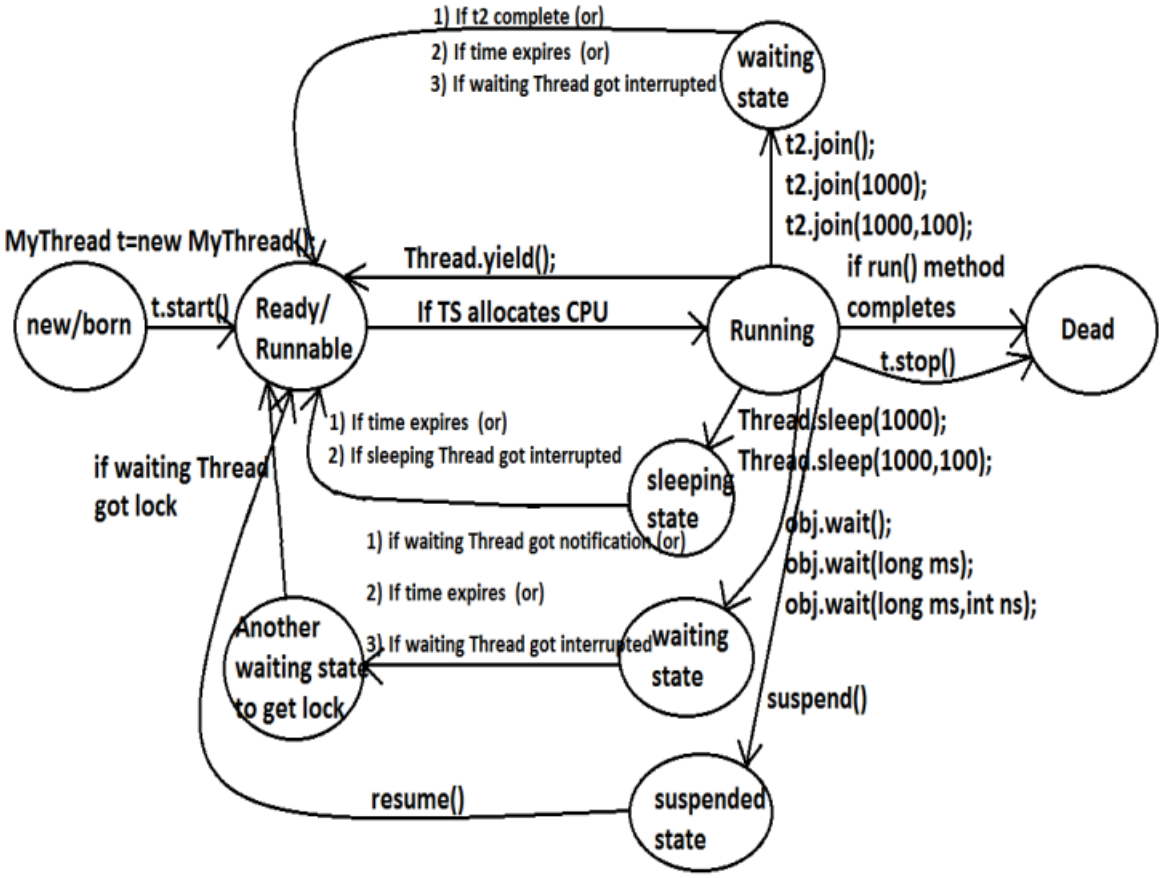
GreenThread Model

- The threads which are managed completely by JVM without taking support for underlying OS, such type of threads are called Green Threads.

Native OS Model

- The Threads which are managed with the help of underlying OS are called Native Threads.
- Windows based OS provide support for Native OS Model .
- Very few OS like SunSolaries provide support for GreenThread Model .
- Anyway GreenThread model is deprecated and not recommended to use .

Life cycle of a Thread:



ThreadGroup:

- Based on the Functionality we can Group Threads into a Single Unit which is Nothing but ThreadGroup
  i.e. ThreadGroup Represents a Set of Threads.
- In Addition a ThreadGroup can Also contains Other SubThreadGroups.
- ThreadGroup Class Present in java.lang Package and it is the Direct Child Class of Object.
- ThreadGroup provides a Convenient Way to Perform Common Operation for all Threads belongs to a Particular Group.
  Eg: Stop All Consumer Threads.
  Suspend All Producer Threads.

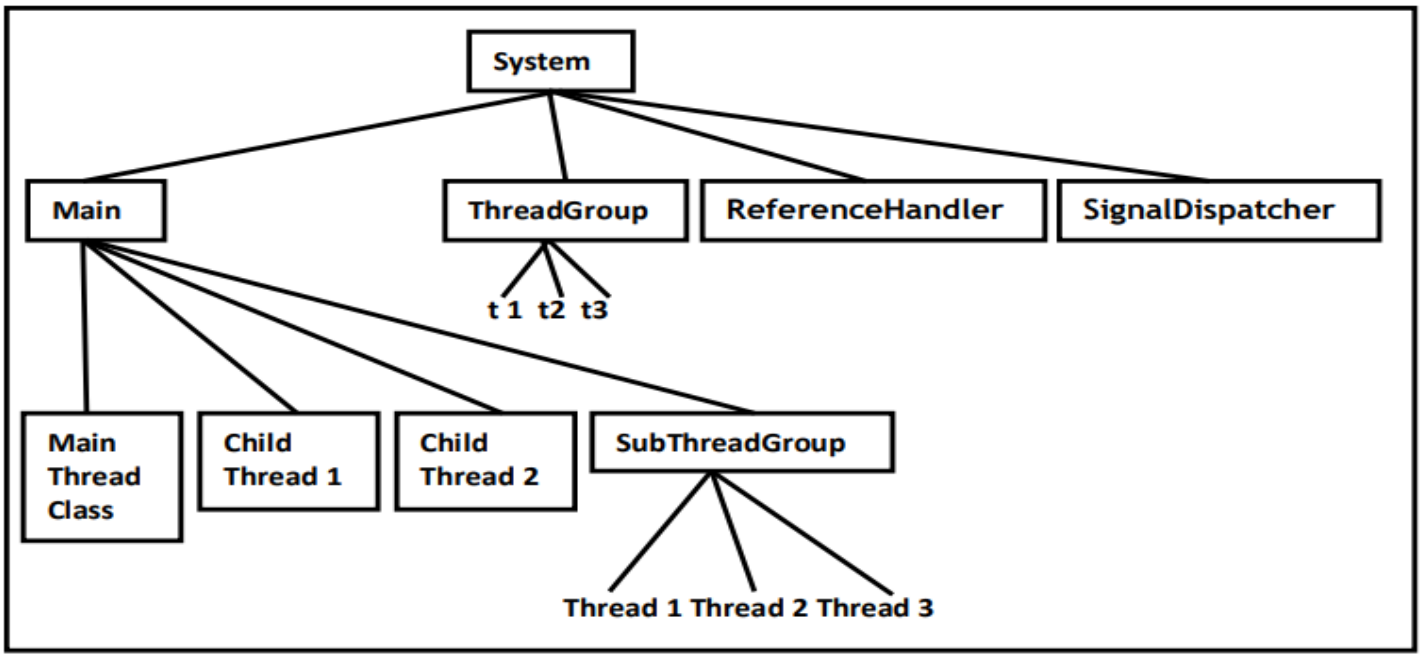
Constructors:

1) ThreadGroup g = new ThreadGroup(String gname);
   Creates a New ThreadGroup.
   The Parent of this New Group is the ThreadGroup of Currently Running Thread.
2) ThreadGroup g = new ThreadGroup(ThreadGroup pg, String gname);
   Creates a New ThreadGroup.
   The Parent of this ThreadGroup is the specified ThreadGroup.

Note:

- In Java Every Thread belongs to Some Group.

- Every ThreadGroup is the Child Group of **System Group** either Directly OR Indirectly. Hence SystemGroup Acts as Root for all ThreadGroup's in Java.
- System ThreadGroup Represents System Level Threads Like ReferenceHandler, SignalDispatcher, Finalizer, AttachListener Etc.



Important Methods of ThreadGroup Class:

1) String getName(); Returns Name of the ThreadGroup.

2) int getMaxPriority(); Returns the Maximum Priority of ThreadGroup.

3) void setMaxPriority();

- To Set Maximum Priority of ThreadGroup.
- The Default Maximum Priority is 10.
- Threads in the ThreadGroup that Already have Higher Priority, Not effected but Newly Added Threads this MaxPriority is Applicable.

4) ThreadGroupgetParent(): Returns Parent Group of Current ThreadGroup.

5) void list(): It Prints Information about ThreadGroup to the Console.

6) int activeCount(): Returns Number of Active Threads Present in the ThreadGroup.

7) int activeGroupCount(): It Returns Number of Active ThreadGroups Present in the Current ThreadGroup.

8) int enumerate(Thread[] t): To Copy All Active Threads of this Group into provided Thread Array. In this Case SubThreadGroup Threads also will be Considered.

9) int enumerate(ThreadGroup[] g): To Copy All Active SubThreadGroups into ThreadGroupArray.

10) boolean isDaemon():

11) void setDaemon(boolean b):

12) void interrupt(): To Interrupt All Threads Present in the ThreadGroup.

13) void destroy(): To Destroy ThreadGroup and its SubThreadGroups.

Java.util.concurrent.locks package:

Problems with Traditional synchronized Key Word

- If a Thread Releases the Lock then which waiting Thread will get that Lock we are Not having any Control on this.
- We can't Specify Maximum waiting Time for a Thread to get Lock so that it will Wait until getting Lock, which May Effect Performance of the System and Causes Dead Lock.
- We are Not having any Flexibility to Try for Lock without waiting.
- There is No API to List All Waiting Threads for a Lock.
- The synchronized Key Word Compulsory we have to Define within a Method and it is Not Possible to Declare Over Multiple Methods.
- To Overcome Above Problems SUN People introduced **java.util.concurrent.locks** Package in 1.5 Version.
- It Also Provides Several Enhancements to the Programmer to Provide More Control on Concurrency.

Lock(I):

- A Lock Object is Similar to Implicit Lock acquired by a Thread to Execute synchronized Method OR synchronized Block
- Lock Implementations Provide More Extensive Operations than Traditional Implicit Locks.

Important Methods of Lock Interface

1. void **lock**();
2. boolean **tryLock**();
3. boolen tryLock(long time, TimeUnit unit);
4. void **lockInterruptedly**();
   Acquired the Lock Unless the Current Thread is Interrupted.
5. void **unlock**(); To Release the Lock.

ReentrantLock

- It implements Lock Interface and it is the Direct Child Class of an Object.
- Reentrant Means a Thread can acquires Same Lock Multiple Times without any Issue.
- Internally ReentrantLock Increments Threads Personal Count whenever we Call lock() and Decrements Counter whenever we Call unlock() and Lock will be Released whenever Count Reaches '0'.

Constructors:

1) ReentrantLockr l = new ReentrantLock();
   Creates an Instance of ReentrantLock.
2) ReentrantLockr l = new ReentrantLock(boolean fairness);
   Creates an Instance of ReentrantLock with the Given Fairness Policy.
   If Fairness is true then Longest Waiting Thread can acquired Lock Once it is Avaiable .
   i.e. if follows First - In First – Out.
   If Fairness is false then we can't give any Guarantee which Thread will get the Lock Once it is Available.

Note: If we are Not specifying any Fairness Property then by Default it is Not Fair.

Important Methods of ReentrantLock

1. void **lock**();
2. boolean **tryLock**();
3. boolean **tryLock**(long l, TimeUnit t);
4. void **lockInterruptedly**();
5. void **unlock**();
6. int **getHoldCount**(); Returns Number of Holds on this Lock by Current Thread.
7. boolean **isHeldByCurrentThread**(); Returns true if and Only if Lock is Hold by Current Thread.
8. int **getQueueLength**(); Returns the Number of Threads waiting for the Lock.
9. Collection **getQueuedThreads**(); Returns a Collection containing Thread Objects which are waiting to get the Lock.
10. boolean **hasQueuedThreads**(); Returns true if any Thread waiting to get the Lock.
11. boolean **isLocked**(); Returns true if the Lock acquired by any Thread.
12. boolean **isFair**(); Returns true if the Lock's Fairness Set to true.
13. Thread **getOwner**(); Returns the Thread which acquires the Lock.

Types of Luck in Java

1) ReentrantLock
2) ReadWriteLock
3) StampedLock
4) SemaphoreLock
5) Codition

1. ReentrantLock:

- **Definition**: `ReentrantLock` is a synchronization primitive in Java that extends the `Lock` interface, **allowing a thread to acquire the lock multiple times.**
- **Usage**: It provides more flexibility and features compared to `synchronized` blocks, such as the ability to specify timeouts for lock acquisition, condition variables, and fairness policies.
- **Real-time Scenario**: Consider a banking application where multiple threads handle transactions. Each transaction involves deducting an amount from the account balance. You can use a `ReentrantLock` to ensure that only one thread updates the account balance at a time to prevent race conditions.

2. ReadWriteLock:

- **Definition**: `ReadWriteLock` is an interface in Java that **provides separate locks for read and write operations on a shared resource.**
- **Usage**: Multiple threads can concurrently read from the resource, but only one thread can write to the resource at a time. This improves concurrency for read-heavy workloads.
- **Real-time Scenario**: In a web application, you have a shared cache storing frequently accessed data. Multiple threads can simultaneously read from the cache to serve requests, but only one thread can update the cache with fresh data at a time, ensuring consistency.

3. StampedLock:

- **Definition**: `StampedLock` is an advanced synchronization mechanism introduced in Java 8, **providing support for optimistic locking and read-write operations with stamped tokens.**
- **Usage**: It offers better performance for read operations compared to `ReadWriteLock` by avoiding context switches for some read operations.
- **Real-time Scenario**: Consider a system where a large number of threads frequently read data from a shared data structure. By using `StampedLock`, you can optimize read performance while still allowing occasional write operations.

4. Semaphore:

- **Definition**: `Semaphore` is a synchronization primitive in Java used to **control the number of concurrent threads accessing a shared resource.**
- **Usage**: It maintains a set of permits, and threads must acquire a permit before accessing the resource. Once acquired, permits are released after the thread completes its task.
- **Real-time Scenario**: In a connection pool of a database, you can use a `Semaphore` to limit the maximum number of concurrent database connections. Each thread representing a client must acquire a permit before using a connection, ensuring that the database doesn't get overloaded.

5. Condition:

- **Definition**: `Condition` is an interface in Java that provides a way for threads to suspend execution until a particular condition is satisfied.
- **Usage**: It is typically used in conjunction with `ReentrantLock` to await or signal specific conditions related to shared resources.
- **Real-time Scenario**: In a producer-consumer scenario, multiple producer threads may produce items, and multiple consumer threads may consume them. You can use a `Condition` to notify consumer threads when items are available for consumption, and to notify producer threads when space becomes available in the buffer.

  Ex: await()
  signal()
  signalAll()

Thread Pools:

- Creating a New Thread for Every Job May Create Performance and Memory Problems.
- To Overcome this we should go for Thread Pool Concept.
- Thread Pool is a Pool of Already Created Threads Ready to do Our Job.
- Java 1.5 Version Introduces Thread Pool Framework to Implement Thread Pools.
- Thread Pool Framework is Also Known as Executor Framework.
- We can Create a Thread Pool as follows

  ExecutorService service = Executors.newFixedThreadPool(3);//Our Choice
- We can Submit a Runnable Job by using submit().

  service.**submit**(job);
- We can ShutdownExecutiorService by using shutdown().

  service.**shutdown**();

Callable and Future:

- In the Case of Runnable Job Thread won't Return anything.
- If a Thread is required to Return Some Result after Execution then we should go for Callable.
- Callable Interface contains Only One Method public Object call() throws Exception.
- If we Submit a Callable Object to Executor then the Framework Returns an Object of Type java.util.concurrent.Future
- The Future Object can be Used to Retrieve the Result from Callable Job.

ThreadLocal(1.2 v):

- We can use ThreadLocal to define local resources which are required for a perticular Thread like DBConnections, counterVariables etc.,
- We can use ThreadLocal to define Thread scope like Servlet Scopes(page,request,session,application).

Atomic

- Atomic operations ensure that shared data is accessed and modified in a thread-safe manner, without the need for explicit locking mechanisms.

1. **AtomicInteger**: This class provides atomic operations on integer variables. It supports atomic increments and decrements, as well as atomic updates using methods like `getAndIncrement()`, `getAndDecrement()`, `getAndAdd(int delta)`, and `compareAndSet(int expect, int update)`.

```
AtomicInteger atomicInt = new AtomicInteger(0);
int incrementedValue = atomicInt.incrementAndGet();
int decrementedValue = atomicInt.decrementAndGet();
```

2. **AtomicBoolean**: As the name suggests, this class provides atomic operations on boolean variables. It supports atomic updates and comparisons using methods like `compareAndSet(boolean expect, boolean update)`.

```
AtomicBoolean atomicBool = new AtomicBoolean(true);
boolean previousValue = atomicBool.compareAndSet(true, false);
```

3. **AtomicLong**: Similar to `AtomicInteger`, this class provides atomic operations on long variables. It supports atomic increments and decrements, as well as atomic updates using methods like `getAndIncrement()`, `getAndDecrement()`, `getAndAdd(long delta)`, and `compareAndSet(long expect, long update)`.

```
AtomicLong atomicLong = new AtomicLong(0L);
long incrementedValue = atomicLong.incrementAndGet();
long decrementedValue = atomicLong.decrementAndGet();
```

4. **AtomicReference**: This class provides atomic operations on reference variables. It supports atomic updates and comparisons using methods like `compareAndSet(V expect, V update)`.

```
AtomicReference<String> atomicRef = new AtomicReference<>("initialValue");
String previousValue = atomicRef.getAndSet("newValue");
```

FAQS

Questions:

1. What is a Thread?
2. Which Thread by default runs in every java program?
   Ans: By default main Thread runs in every java program.
1. What is the default priority of the Thread?
2. How can you change the priority number of the Thread?
3. Which method is executed by any Thread?
   Ans: A Thread executes only public void run() method.
4. How can you stop a Thread which is running?
5. Explain the two types of multitasking?
   Ans: Thread Based & Process Based
6. What is the difference between a process and a Thread?
7. What is Thread scheduler?
8. Explain the synchronization of Threads?
9. What is the difference between synchronized block and synchronized keyword?
10. What is Thread deadlock? How can you resolve deadlock situation?
11. Which methods are used in Thread communication?
12. What is the difference between notify() and notifyAll() methods?
13. What is the difference between sleep() and wait() methods?
14. Explain the life cycle of a Thread?
15. What is daemon Thread ?
16. Explain about synchronized keyword and its advantages and disadvantages?

17. What is object lock and when a Thread required?
18. What is class level lock and when a Thread required?
19. What is the difference between object lock and class level lock?
20. While a Thread executing a synchronized method on the given object is the remaining Threads are allowed to execute other synchronized methods simultaneously on the same object?
    Ans: No.
21. What is synchronized block and explain its declaration?
22. What is the advantage of synchronized block over synchronized method?
23. Is a Thread can hold more than one lock at a time?
    Ans: Yes, up course from different objects.
24. What is synchronized statement?
    Ans: The statements which present inside synchronized method and synchronized block are called synchronized statements. [Interview people created terminology].
25. What Is Multi Tasking?
26. What Is Multi ThreadingAnd Explain Its Application Areas?
27. What Is Advantage Of Multi Threading?
28. When Compared With C++ What Is The Advantage In Java With Respect To Multi Threading?
29. In How Many Ways We Can Define A Thread?
30. Among Extending Thread And Implementing Runnable Which Approach Is Recommended?
31. Difference Between t.start() And t.run()?
32. Explain About Thread Scheduler?
33. If We Are Not Overriding run() What Will Happen?
34. Is It Possible Overloading Of run()?
35. Is It Possible To Override a start() And What Will Happen?
36. Explain Life Cycle Of A Thread?
37. What Is The Importance Of Thread Class start()?
38. After Starting A Thread If We Try To Restart The Same Thread Once Again What Will Happen?
39. Explain Thread Class Constructors?
40. How To Get And Set Name Of A Thread?
41. Who Uses Thread Priorities?
42. Default Priority For Main Thread?
43. Once We Create A New Thread What Is Its Priority?
44. How To Get Priority From Thread And Set Priority To A Thread?
45. If We Are Trying To Set Priority Of Thread As 100, What Will Happen?
46. If 2 Threads Having Different Priority Then Which Thread Will Get Chance First For Execution?
47. If 2 Threads Having Same Priority Then Which Thread Will Get Chance First For Execution?
48. How We Can Prevent Thread From Execution?
49. What Is yield() And Explain Its Purpose?
50. Is Join Is Overloaded?
51. Purpose Of sleep()?
52. What Is synchronized Key Word? Explain Its Advantages And Disadvantages?
53. What Is Object Lock And When It Is Required?
54. What Is A Class Level Lock When It Is Required?
55. While A Thread Executing Any Synchronized Method On The Given Object Is It Possible To Execute Remaining Synchronized Methods On The Same Object Simultaneously By Other Thread?
56. Difference Between Synchronized Method And Static Synchronized Method?
57. Advantages Of Synchronized Block Over Synchronized Method?
58. What Is Synchronized Statement?
59. How 2 Threads Will Communicate With Each Other?
60. wait(), notify(), notifyAll() Are Available In Which Class?
61. Why wait(), notify(), notifyAll() Methods Are Defined In Object Instead Of Thread Class?
62. Without Having The Lock Is It Possible To Call wait()?
63. 39)If A Waiting Thread Gets Notification Then It Will Enter Into Which State?
64. In Which Methods Thread Can Release Lock?

65. Explain wait(), notify() and notifyAll()?
66. Difference Between notify() and notifyAll()?
67. Once A Thread Gives Notification Then Which Waiting Thread Will Get A Chance?
68. How A Thread Can Interrupt Another Thread?
69. What Is Deadlock? Is It Possible To Resolve Deadlock Situation?
70. Which Keyword Causes Deadlock Situation?
71. How We Can Stop A Thread Explicitly?
72. Explain About suspend() And resume()?
73. What Is Starvation And Explain Difference Between Deadlock and Starvation?
74. What Is Race Condition?
75. What Is Daemon Thread? Give An Example Purpose Of Daemon Thread?
76. How We Can Check Daemon Nature Of A Thread? Is It Possible To Change Daemon Nature Of A Thread? Is Main Thread Daemon OR Non-Daemon?
77. Explain About ThreadGroup?
78. What Is ThreadLocal ?

Collection

Collection interface Methods :

1) boolean add(Object o);
2) boolean addAll(Collection c);
3) boolean remove(Object o);
4) boolean removeAll(Object o);
5) boolean retainAll(Collection c); //To remove all objects except those present in c.
6) Void clear();
7) boolean contains(Object o);
8) boolean containsAll(Collection c);
9) boolean isEmpty();
10) Int size();
11) Object[] toArray();
12) Iterator iterator();

Default Capacities

1. ArrayList: The default initial capacity is 10.
2. HashMap: The default initial capacity is 16.
3. HashSet: The default initial capacity is 16.
4. LinkedHashMap: The default initial capacity is 16.
5. LinkedHashSet: The default initial capacity is 16.
6. LinkedList: Since `LinkedList` does not have a fixed capacity, there is no default initial capacity.
7. PriorityQueue: Since `PriorityQueue` does not have a fixed capacity, there is no default initial capacity.
8. Stack: Since `Stack` is based on `Vector`, its default initial capacity is 10.
9. TreeMap: Since `TreeMap` is based on a red-black tree, it does not have a fixed capacity, and there is no default initial capacity.

10. TreeSet: Since `TreeSet` is based on `TreeMap`, it does not have a fixed capacity, and there is no default initial capacity.

11. Vector: The default initial capacity is 10.

ArrayList:

1. The underlying data structure is resizable array (or) growable array.
2. Duplicate objects are allowed.
3. Insertion order preserved.
4. Heterogeneous objects are allowed.(except TreeSet , TreeMap every where heterogenious objects are allowed)
5. Null insertion is possible.
6. ArrayList is nonsynchronize in nature .

Constructors:

1) ArrayList a=new ArrayList();
2) ArrayList a=new ArrayList(int initialcapacity);
3) ArrayList a=new ArrayList(collection c);

- Usually we can use collection to hold and transfer objects from one tier to another tier. To provide support for this requirement every Collection class already implements Serializable and Cloneable interfaces.
- ArrayList and Vector classes implements RandomAccess interface so that any random element we can access with the same speed. Hence ArrayList is the best choice of "**retrival operation**".
- RandomAccess interface present in util package and doesn't contain any methods. It is a **marker interface**.

Marker interface

- An interface that does not contain methods, fields, and constants is known as marker interface.
- In other words, an empty interface is known as marker interface or tag interface.
- It delivers the run-time type information about an object.
- It is the reason that the JVM and compiler have additional information about an object.
- The Serializable and Cloneable and RandomAccess interfaces are the example of marker interface.

How to make an ArrayList as Synchronize .

1) using **synchronizedList**();
   - We can make an ArrayList synchronized by wrapping it with the Collections.synchronizedList() method .
   - But it will work in case of add, remove and In case of traverse or fetch we need to create one more synchronized block.
   - if we are iterating over a synchronized list returned by Collections.synchronizedList(), and another thread tries to modify the list by adding or removing elements, it may lead to a ConcurrentModificationException or other unexpected behavior.

Ex::

```
List<String> synchronizedList = Collections.synchronizedList(new ArrayList<>());
      synchronizedList.add("A");
      synchronizedList.add("B");
      synchronizedList.add("C");
// Synchronized block for iteration
synchronized (synchronizedList) {
   Iterator<String> iterator = synchronizedList.iterator();
```

```
   while (iterator.hasNext()) {
      String element = iterator.next();
      // Do something with the element
   }
}
```

2) **CopyOnWriteArrayList**

- CopyOnWriteArrayList is another option in Java for creating a thread-safe list.
- It is a concurrent collection class introduced in Java 5.
- It provides thread safety without the need for explicit synchronization .

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
    // Add elements to the list
    list.add("A");
    list.add("B");
    list.add("C");
        Iterator<String> iterator = list.iterator();
  while (iterator.hasNext()) {
    String element = iterator.next();
    // Do something with the element
  }
```

FailSafe and FailFast

- Using iterations we can traverse over the collections objects. The iterators can be either fail-safe or fail-fast.
- Fail-fast iterators throw an exception(**ConcurrentModificationException**) if the collection is modified while iterating over it.

Example:

```
ArrayList<Integer> integers = new ArrayList<>();
integers.add(1);
integers.add(2);
Iterator<Integer> itr = integers.iterator();
while (itr.hasNext()) {
   Integer a = itr.next();
   integers.remove(a);}
```

- As arrayLists are fail-fast above code will throw an exception. First a will have value = 1, and then 1 will be removed in same iteration. Next when a will try to get next(), as the modification is made to the list, it will throw an exception here.

Fail-safe

- Fail-safe iterators means they will not throw any exception even if the collection is modified while iterating over it.
- If we use an fail-safe collection e.g. CopyOnWriteArrayList then no exception will occur .

Example:

```
List<Integer> integers = new CopyOnWriteArrayList<>();
integers.add(1);
integers.add(2);
integers.add(3);
Iterator<Integer> itr = integers.iterator();
while (itr.hasNext()) {
   Integer a = itr.next();
   integers.remove(a);
}
```

Several ways to iterate over an ArrayList in Java

1) Using a **for loop**
2) Using an enhanced for loop **(for-each** loop)
3) Using an **Iterator**

```
Iterator<String> iterator = list.iterator();

while (iterator.hasNext()) {
   String element = iterator.next();
   // Do something with the element
}
```

4) Using a **ListIterator**

```
ListIterator<String> listIterator = list.listIterator();
while (listIterator.hasNext()) {
   String element = listIterator.next();
   // Do something with the element
}
```

5) **Using forEach()** method with lambda expression or method reference (Java 8 and later)

```
list.forEach(element -> {
   // Do something with the element
});
// Or using method reference
list.forEach(System.out::println);
```

Iterator **Vs** **ListIterator**

▪ Only one Way direction	▪ Both forward and backward
▪ Hasnext(), next() , remove()	▪ Hasprevious(), previous() , add() , set()
▪ Can be Used arraylist , linkedlist .	▪ Can be Used in arraylist, linkedlist, and Vector.

LinkedList

1) The underlying data structure is double LinkedList
2) If our frequent operation is insertion (or) deletion in the middle then LinkedList is the best choice.
3) If our frequent operation is retrieval operation then LinkedList is worst choice.
4) Duplicate objects are allowed.
5) Insertion order is preserved.
6) Heterogeneous objects are allowed.
7) Null insertion is possible.
8) Implements Serializable and Cloneable interfaces but not RandomAccess.

- Usually we can use LinkedList to implement Stacks and Queues.

Methods.

1) void addFirst(Object o);
2) void addLast(Object o);
3) Object getFirst();
4) Object getLast();

5) Object removeFirst();
6) Object removeLast();

How to make an LinkedList as Synchronize .

- using **Collections.synchronizedList** ();
- We can make an LinkedList synchronized by wrapping it with the Collections.synchronizedList() method .
- But it will work in case of add, remove and In case of traverse or fetch we need to create one more synchronized block like ArrayList. Here copyOnWrite is not there .

Vector:

1) The underlying data structure is resizable array (or) growable array.
2) Duplicate objects are allowed.
3) Insertion order is preserved.
4) Heterogeneous objects are allowed.
5) Null insertion is possible.
6) Implements Serializable, Cloneable and RandomAccess interfaces.

Vector v=new Vector();

- Creates an empty Vector object with default initial capacity 10.
- Once Vector reaches its maximum capacity then a new Vector object will be created with double capacity. That is "newcapacity=currentcapacity*2".
- Every **method** present in Vector is **synchronized** and hence Vector is Thread safe.

Difference between Vector and ArrayList :

1. Synchronization
2. Performance
3. Resize policy - ArrayList increase 50% where as Vector currentcapacity*2
4. Legacy - ArrayList 1.2v , Vector 1.0

Stack:

- It is the child class of Vector.
- Whenever last in first out(LIFO) order required then we should go for Stack.

Constructor:

It contains only one constructor.
Stack s= new Stack();

Methods:

1. Object push(Object o); To insert an object into the stack.

2. Object pop(); To remove and return top of the stack.

3. Object peek(); To return top of the stack without removal.

4. boolean empty(); Returns true if Stack is empty.

5. Int search(Object o); Returns offset if the element is available otherwise returns "-1"

The 3 cursors of java:

- If we want to get objects one by one from the collection then we should go for cursor.
  There are 3 types of cursors available in java. They are:
  1. Enumeration
  2. Iterator

3. ListIterator

Enumeration:

1) We can use Enumeration to get objects one by one from the legacy collection objects.
2) We can create Enumeration object by using elements() method.

```
public Enumeration elements();
Enumeration e=v.elements();
using Vector Object
```

3) Enumeration interface defines the following two methods
   1. public boolean hasMoreElements();
   2. public Object nextElement();

Limitations of Enumeration:

1) We can apply Enumeration concept only for legacy classes and it is not a universal cursor.
2) By using Enumeration we can get only read access and we can't perform remove operations.
3) To overcome these limitations sun people introduced Iterator concept in 1.2v.

Iterator:

1) We can use Iterator to get objects one by one from any collection object.
2) We can apply Iterator concept for any collection object and it is a universal cursor.
3) While iterating the objects by Iterator we can perform both read and remove operations.

We can get Iterator object by using iterator() method of Collection interface.

```
public Iterator iterator();
Iterator itr=c.iterator();
```

- Iterator interface defines the following 3 methods.
  1. public boolean hasNext();
  2. public object next();
  3. public void remove();

Limitations of Iterator:

1) Both enumeration and Iterator are single direction cursors only. That is we can always move only forward direction and we can't move to the backward direction.
2) While iterating by Iterator we can perform only read and remove operations and we can't perform replacement and addition of new objects.
3) To overcome these limitations sun people introduced listIterator concept.

ListIterator:

1) ListIterator is the child interface of Iterator.
2) By using listIterator we can move either to the forward direction (or) to the backward direction that is it is a bi-directional cursor.
3) While iterating by listIterator we can perform replacement and addition of new objects in addition to read and remove operations

By using listIterator method we can create listIterator object.

```
public ListIterator listIterator();
ListIterator itr=l.listIterator();
(l is any List object)
```

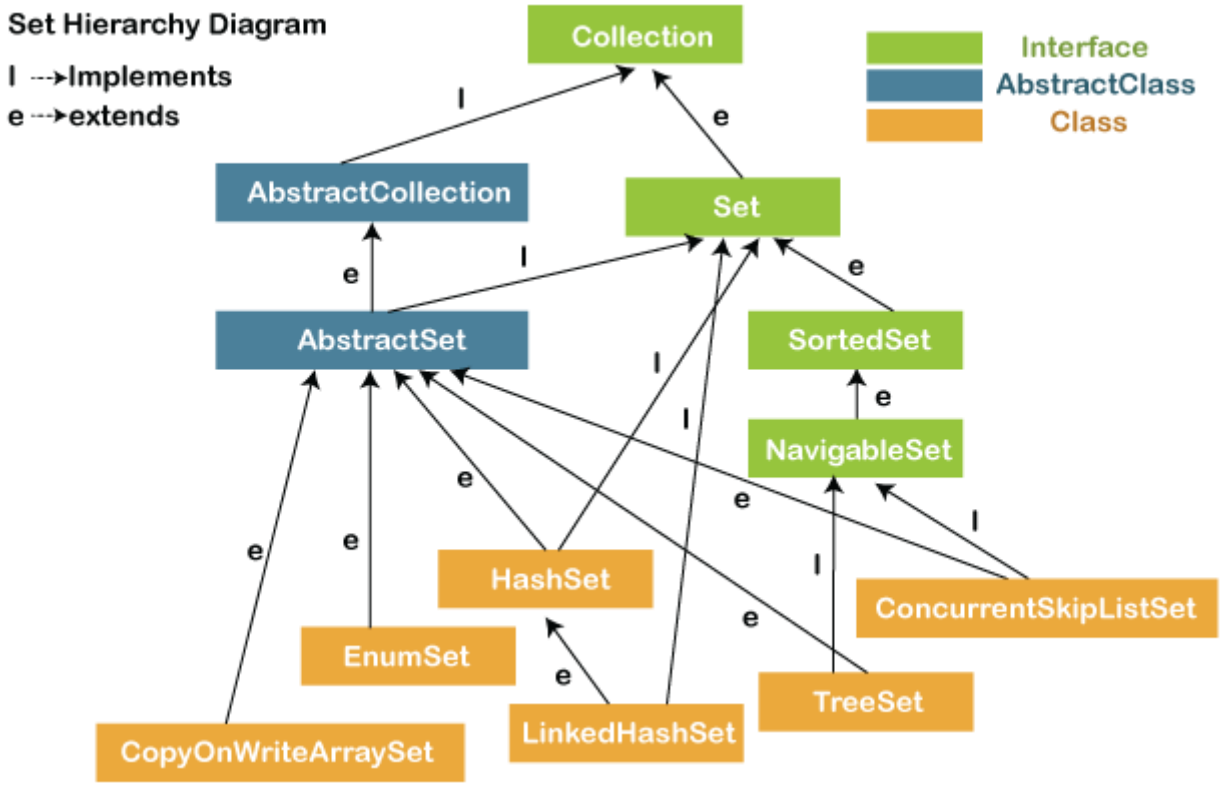
ListIterator interface defines the following 9 methods.

1. public boolean hasNext();
2. public Object next(); forward
3. public int nextIndex();
4. public boolean hasPrevious();

5. public Object previous(); backward
6. public int previousIndex();
7. public void remove();
8. public void set(Object new);
9. public void add(Object new);

Set interface:

- It is the child interface of Collection.
- If we want to represent a group of individual objects as a single entity where duplicates are not allow and insertion order is not preserved then we should go for Set interface.
- Set interface does not contain any new method we have to use only Collection interface methods.



HashSet

1) The underlying data structure is Hashtable.
2) Insertion order is not preserved and it is based on hash code of the objects.
3) Duplicate objects are not allowed.
4) If we are trying to insert duplicate objects we won't get compile time error and runtime error add() method simply returns false.
5) Heterogeneous objects are allowed.
6) Null insertion is possible.(only once)
7) Implements Serializable and Cloneable interfaces but not RandomAccess.
8) HashSet is best suitable, if our frequent operation is "Search".

Constructors:

1) HashSet h=new HashSet();

- Creates an empty HashSet object with default initial capacity 16 and default fill ratio 0.75(fill ratio is also known as load factor).

2) HashSet h=new HashSet(int initialcapacity);

- Creates an empty HashSet object with the specified initial capacity and default fill ratio 0.75.

3) HashSet h=new HashSet(int initialcapacity,float fillratio);

4) HashSet h=new HashSet(Collection c);

Note : After filling how much ratio new HashSet object will be created , The ratio is called "**FillRatio**" or "**LoadFactor**".

Making a HashSet Synchronized in Java

- By Default HashSet is not Synchronized .
- Using Following two ways

1. Collections.synchronizedSet();

Ex:

```
HashSet<String> set = new HashSet<>();
Set<String> synchronizedSet = Collections.synchronizedSet(set);
```

2. CopyOnWriteArraySet();

Ex:

```
CopyOnWriteArraySet<String> set = new CopyOnWriteArraySet<>();
```

Lets Understand Internal Working of HashSet :

As HashSet uses Undeline DS HashTable Lets Discuss HashTable :

HashTable :

- A Hash table is defined as a data structure used to insert, look up, and remove key-value pairs quickly.
- It operates on the hashing concept, where each key is translated by a hash function into a distinct index in an array.
- The index functions as a storage location for the matching value. In simple words, it maps the keys with the value.
- The key feature of a hash table is its ability to provide constant-time performance (O(1)) for basic operations like insertion, deletion, and lookup, on average.
- The default size is 16 .
- This makes hash tables ideal for scenarios where fast access to data based on a key is required.
- The load factor in hash tables is a measure of how full the hash table is. It is defined as the ratio of the number of elements stored in the hash table to the number of slots (buckets) available in the underlying array. The load factor is denoted by the symbol λ (lambda).

  Load Factor (λ) = Number of elements / Number of slots

- Collision handling is a critical aspect of hash table implementation, as collisions occur when two or more keys hash to the same slot in the underlying array.

Each Value Stored as Key-Value Pair:

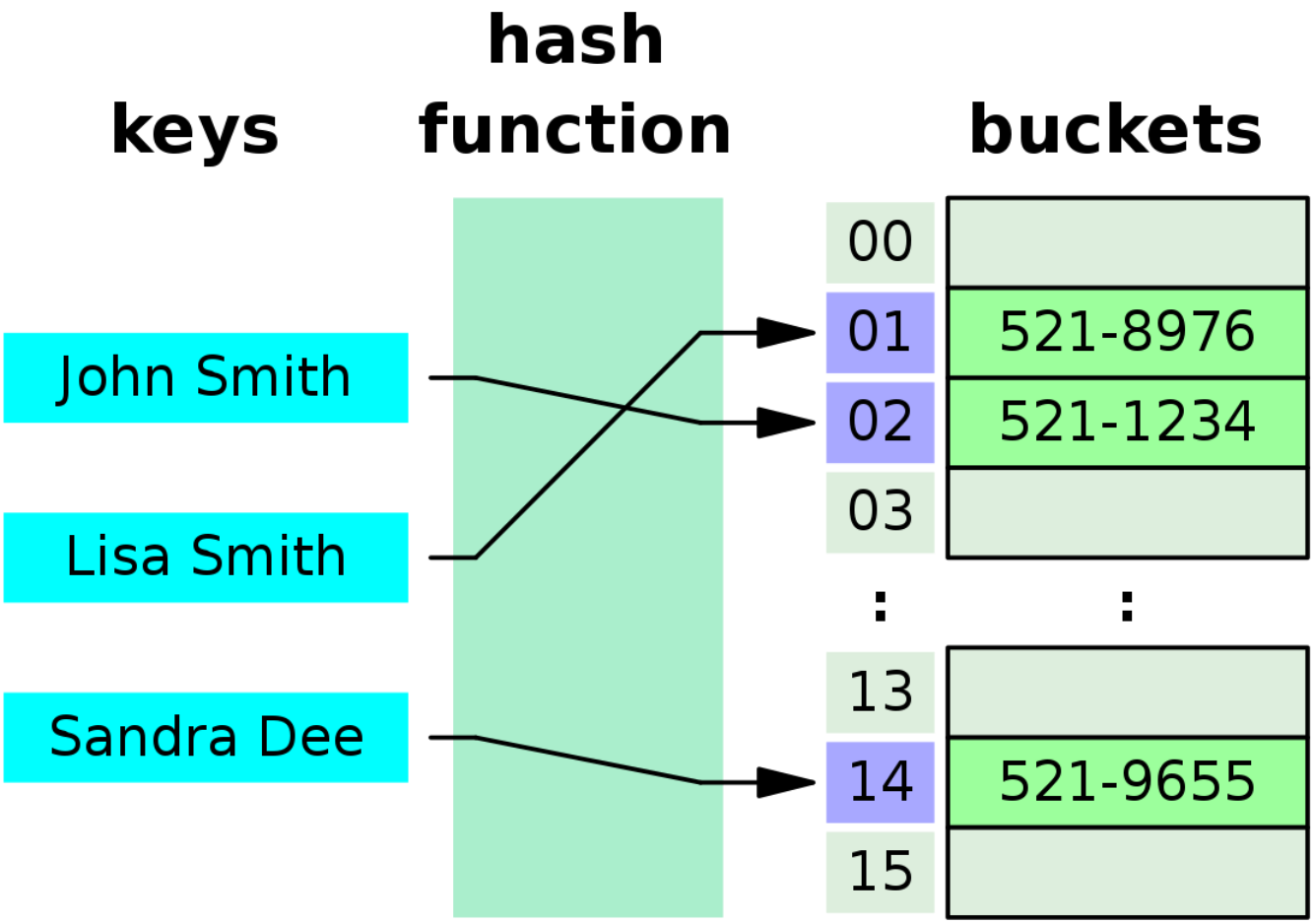
- In both hash tables and maps, each entry is typically stored as a key-value pair.

Components of Each Entry:

- **Hash**: The hash code of the key is used to determine the bucket where the entry will be stored.
- **Key**: Represents the element entered into the data structure.
- **Value**: In the case of a set, it's typically PRESENT to indicate the presence of the key. In maps, both the key and the corresponding value are stored.
- **Next**: In the case of hash collisions (when multiple entries hash to the same bucket), this field holds the reference to the next entry in a linked list or similar structure.

Handling Collisions:

- When multiple entries hash to the same bucket, they are managed using techniques like chaining (linked lists) or open addressing.



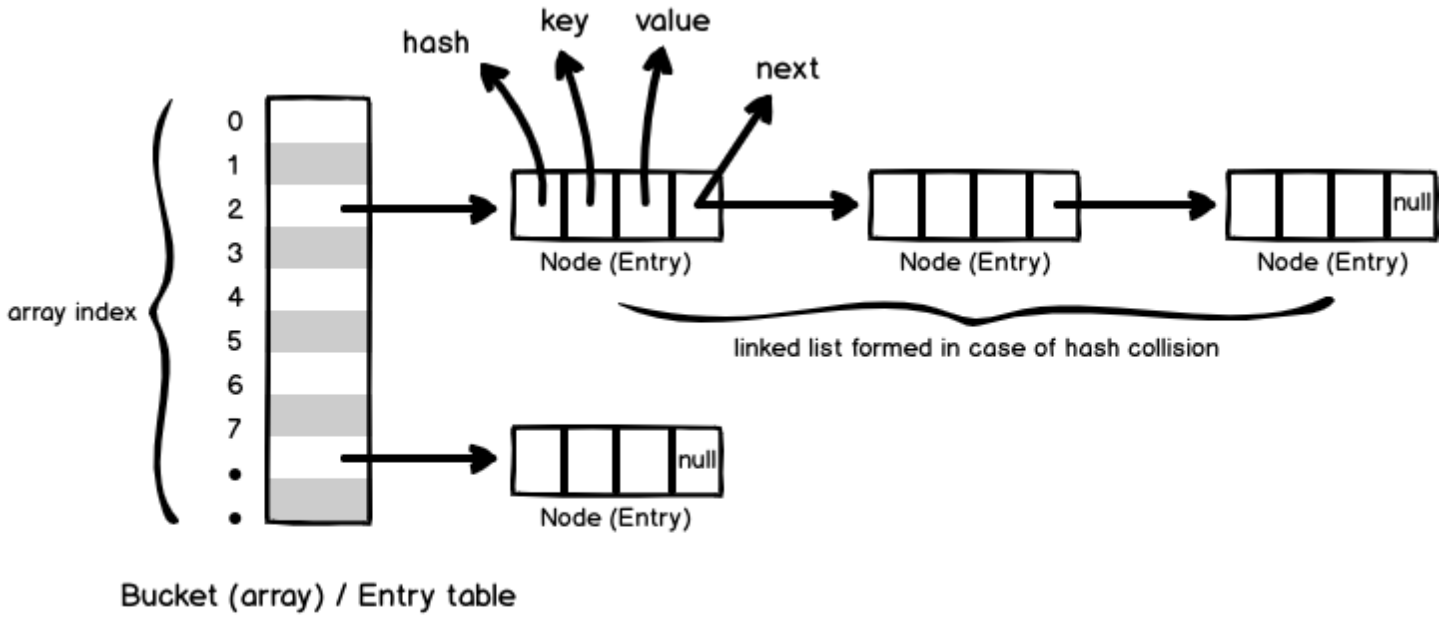
HashSet Internals

- HashSet uses HashMap internally to store data in the form of Key-Value pair.
- Elements are stored using ***hashing technique***, therefore elements are not stored in an ordered fashion and the elements will be returned in random order.
- Elements are stored in the form of key-value pair (internally by HashMap) where key will the actual element value and value will be the ***PRESENT*** Constant.
- In order to understand HashSet internal behaviour , one should understand the HashMap internals. This is because HashSet internally uses the HashMap behaviour.
- The moment you create a HashSet the default constructor will get called and internally a backing HashMap object gets created automatically.

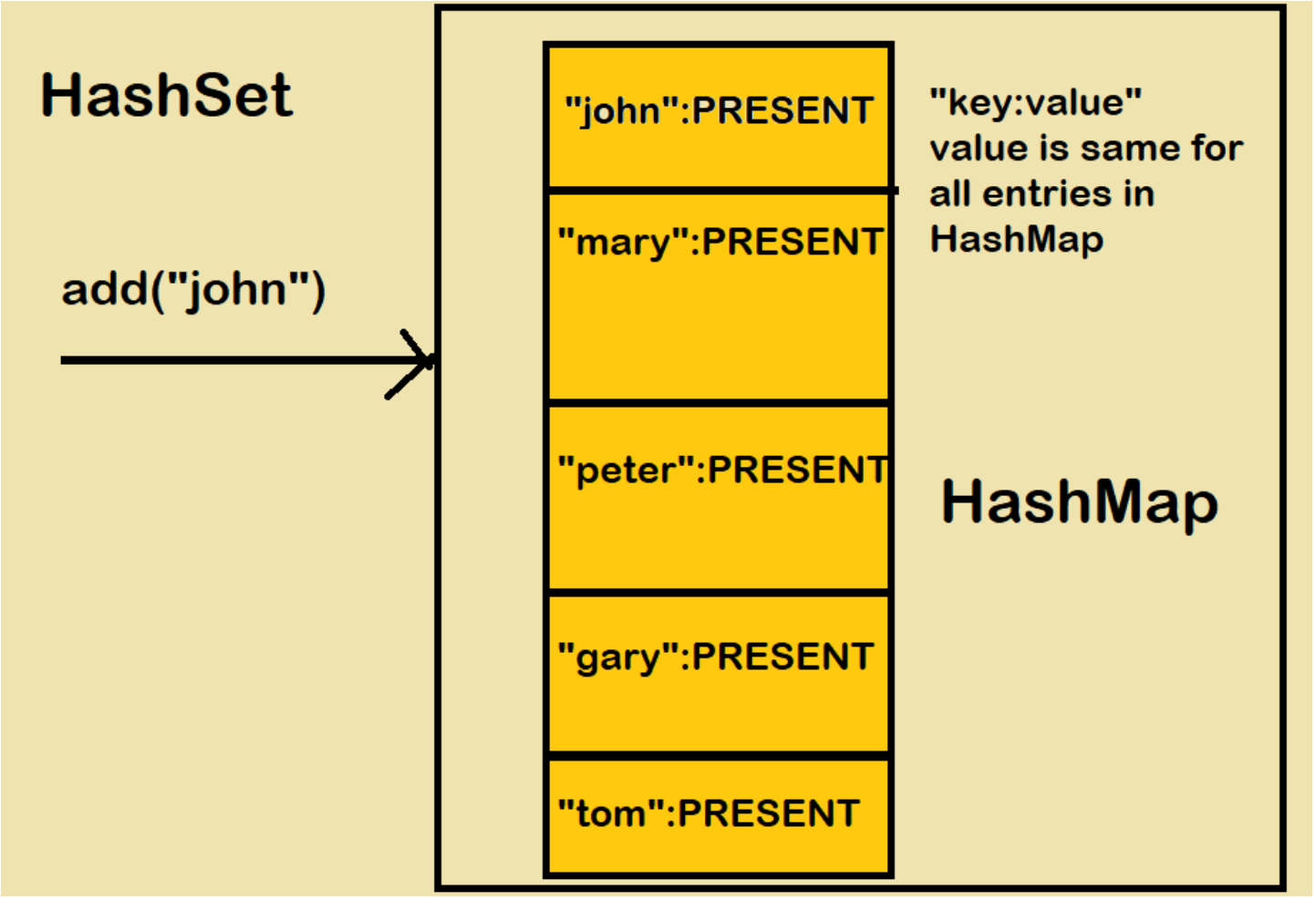
  Set<String> hs = new HashSet<String>();

- In HashSet class, a map instance can be created at class level like below

  private transient HashMap<E,Object> map;



HashMap



LinkedHashSet

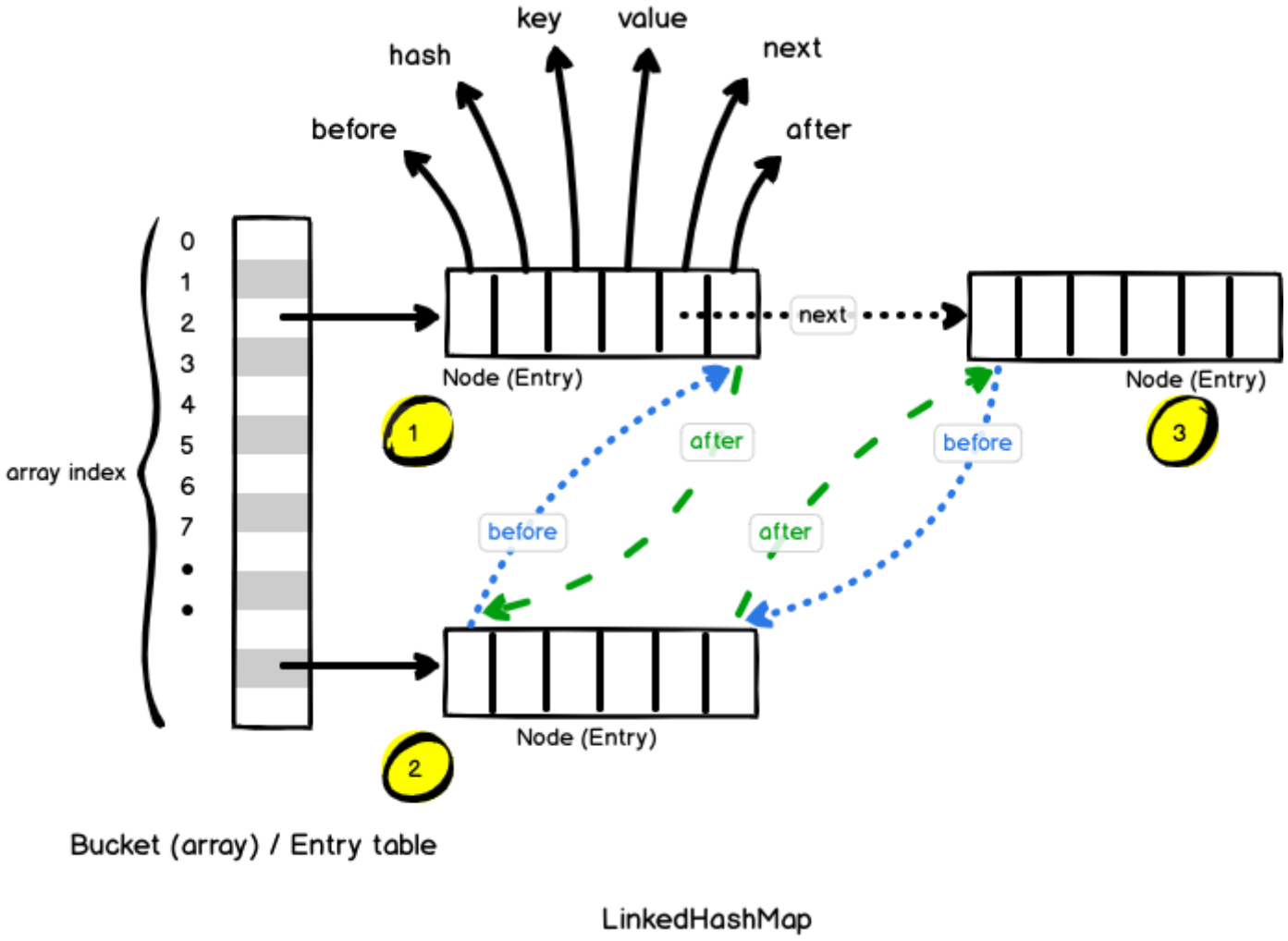
- It is the child class of HashSet.

- LinkedHashSet is exactly same as HashSet except the following differences.
- The underlying data structure is a combination of LinkedList and Hashtable .
- Insertion order is preserved.

Note: LinkedHashSet and LinkedHashMap commonly used for implementing "cache applications" where insertion order must be preserved and duplicates are not allowed.

Ways to iterate over a LinkedHashSet:

1. Using Enhanced for Loop (for-each loop)
2. Using Iterator
3. Using forEach() method with Lambda Expression (Java 8 and later)



LinkedHashMap

SortedSet:

- It is child interface of Set.
- If we want to represent a group of "unique objects" where duplicates are not allowed and all objects must be inserting according to some sorting order then we should go for SortedSet interface.
- That sorting order can be either default **natural sorting** (or) customized sorting order .
- Accept only Homogeneous Data .

SortedSet interface define the following 6 specific methods.

1. Object first();

2. Object last();
3. SortedSet headSet(Object obj);
   - Returns the SortedSet whose elements are <obj.
4. SortedSet tailSet(Object obj);
   - It returns the SortedSet whose elements are >=obj.
5. SortedSet subset(Object o1,Object o2);
   - Returns the SortedSet whose elements are >=o1 but <o2.
6. **Comparator comparator**();
   - Returns the Comparator object that describes underlying sorting technique.
   - If we are following default natural sorting order then this method returns null.

TreeSet

JAVA 8 NEW FEATURES

1) Lambda Expression

2) Functional Interfaces

3) Default And Static Methods

4) Predicates

5) Functions

6) Consumer

7) Supplier

8) Double colon operator(::)

9) Stream API

10) Date and Time API

Lambda Expressions

- Lambda expression helps us to write our code in functional style.
- Lambda Expression is just an anonymous(nameless) function. That means the function which doesn't have the name,return type and access modifiers.
- Lambda Expression also known as anonymous functions or closures.
- It provides a clear and concise way to implement SAM interface(Single Abstract Method) by using an expression.

Syntax :

```
(argument-list) -> {body}
```

Ex::

```
() -> {
//Body of no parameter lambda
}
```

Functional Interfaces

- A Functional Interface is an interface that has exactly one abstract method.
- It can have any number of default and static methods. It can also declare methods of object class.
- Functional interfaces are also known as Single Abstract Method Interfaces (SAM Interfaces).
- The @FunctionalInterface annotation prevents abstract methods from being accidentally added to functional interfaces.
- Inside functional interface in addition to single Abstract method(SAM) we write any number of default and static methods.

Ex::

java.lang. Runnable is a fantastic example of a functional interface since it has one abstract method, run ().

1) Runnable - It contains only run() method

2) Comparable - It contains only compareTo() metho

3) ActionListener - It contains only actionPerformed()

4) Callable - It contains only call()method

Ex::

```
@FunctionalInterface
interface sayable{
   void say(String msg);
}
public class FunctionalInterfaceExample implements sayable{
   public void say(String msg){
      System.out.println(msg);
   }
   public static void main(String[] args) {
      FunctionalInterfaceExample fie = new FunctionalInterfaceExample();
      fie.say("Hello there");
   }
}
```

Anonymous inner classes vs Lambda Expressions

- Wherever we are using anonymous inner classes there may be a chance of using Lambda expression to reduce length of the code and to resolve complexity.

Ex::

```
interface MyInterface {
   void myMethod();
}

// Anonymous inner class
MyInterface anonymousInnerClass = new MyInterface() {
   @Override
   public void myMethod() {
      System.out.println("Anonymous Inner Class");
   }
};

// Lambda expression

MyInterface lambda = () -> System.out.println("Lambda Expression");
```

What are the advantages of Lambda expression?

- We can reduce length of the code so that readability of the code will be improved.
- We can resolve complexity of anonymous inner classes.
- We can provide Lambda expression in the place of object.
- We can pass lambda expression as argument to methods.

Default And Static Methods

- Every variable declared inside interface is always public static final whether we are declaring or not.

- But from 1.8 version onwards in addition to these, we can declare default concrete methods also inside interface,which are also known as defender methods.
- We can declare default method with the keyword “default” .
- Interface static methods by-default not available to the implementation classes hence by using implementation class reference we can’t call interface static methods.we should call interface static methods by using interface name.

Predicates

- A predicate is a function with a single argument and returns boolean value.
- To implement predicate functions injava,oracle people introduced Predicate interface in 1.8 version(i.e.,Predicate<T>).
- Predicate interface present in java.util.function package.
- It’s a functional interface and it contains only one method i.e., test()

Syntax::

```
interface Predicate<T> {
 public boolean test(T t);
}
```

1) Write a predicate to check the length of given string is greater than 3 or not.

```
Predicate<String> p = s → (s.length() > 3);

System.out.println (p.test(“rvkb”)); true

System.out.println (p.test(“rk”)); false
```

2) write a predicate to check whether the given collection is empty or not.

```
Predicate<collection> p = c → c.isEmpty();
```

Function

- Functions are exactly same as predicates except that functions can return any type of result but function should(can)return only one value and that value can be any type as per our requirement.
- To implement functions oracle people introduced Function interface in 1.8version.
- Function interface present in java.util.function package.
- Functional interface contains only one method i.e., apply()

Syntax::

```
interface function(T,R) {
 public R apply(T t);
}
```

Supplier Interface:

- The Supplier<T> interface represents a supplier of results. It has a single abstract method get() that returns a value of type T with no input arguments.

Syntax::

```
@FunctionalInterface
public interface Supplier<T> {
   T get();
}
```

Consumer Interface:

- The Consumer<T> interface represents an operation that accepts a single input argument of type T and returns no result. It has a single abstract method accept(T t).

Syntax::

```
@FunctionalInterface
public interface Consumer<T> {
   void accept(T t);
}
```

Method and Constructor references by using ::(double colon)operator

- Method references provide a way to simplify lambda expressions when calling existing methods or constructors.
- The :: operator, also known as the method reference operator, is used to reference methods or constructors without invoking them.

There are several types of method references:

- Reference to a Static Method: ClassName::staticMethodName
- Reference to an Instance Method of a Particular Object: objectInstance::instanceMethodName
- Reference to an Instance Method of an Arbitrary Object of a Particular Type: ClassName::instanceMethodName
- Reference to a Constructor: ClassName::new

Streams

- if we want to process a group of objects from the collection then we should go for streams.
- we can create a stream object to the collection by using stream()method of Collection interface.
- stream() method is a default method added to the Collection in 1.8 version.

- **Stream Source**: Streams can be created from various sources such as collections, arrays, or by using Stream factory methods.
- **Intermediate Operations**: Intermediate operations are operations that can be chained together to process the data in the stream. These operations are lazy, meaning they do not produce a result until a terminal operation is invoked. Examples of intermediate operations include filter, map, sorted, distinct, etc.
- **Terminal Operations**: Terminal operations are operations that produce a result or side-effect, and they mark the end of a stream pipeline. Examples of terminal operations include forEach, collect, reduce, count, min, max, etc.